Agentic CCP: Architecture Deep Dive and Competitive Research

Nov 9, 2026 · @Claude

Bottom line. In design, Agentic CCP leads the documented competitors on evidence history and on limits for what agents may do: a write-once raw zone, append-only verdicts, bitemporal state, "models extract, code decides", and approvals pinned to a hash of what was approved (C). It is not yet a product: it is teaching-scale by its own spec, its code has verified defects inside those strengths (a typo can approve, the citation check can mark a false pass as verified, and a crash can lose a verdict), and no AI tier has been measured on a real model (D). Keep the core and rebuild the edges. On day 1, push the repo to a private remote so the CI it already has runs on every change. Then spend about nine to twelve weeks (17–24 engineer-weeks, estimated) fixing the verified defects and running the first evals on real models; until that is done, CCP should not handle real customer data or rely on a real model's output. After that, build what a first production customer needs: connectors that record whether each pull was complete, durable execution, signed-in reviewers, an API and a cloud deployment (Section F).

Scope. Agentic CCP (`agentic` at `3386899` , built from `spec.md` v1.3): its code, migrations, tests, fixtures, CI workflows and local eval runs, read on 9 Nov 2026. Vanta, Drata, Secureframe, Thoropass, Sprinto, Scrut, AI-native entrants and open-source GRC projects were researched from their own docs, changelogs, job postings, subprocessor lists and, where it is public, code. Section G is the executive summary; D holds the findings in full. A companion document, Agentic CCP: Target Architecture for AI-Native Compliance, designs CCP's target architecture from the same evidence.

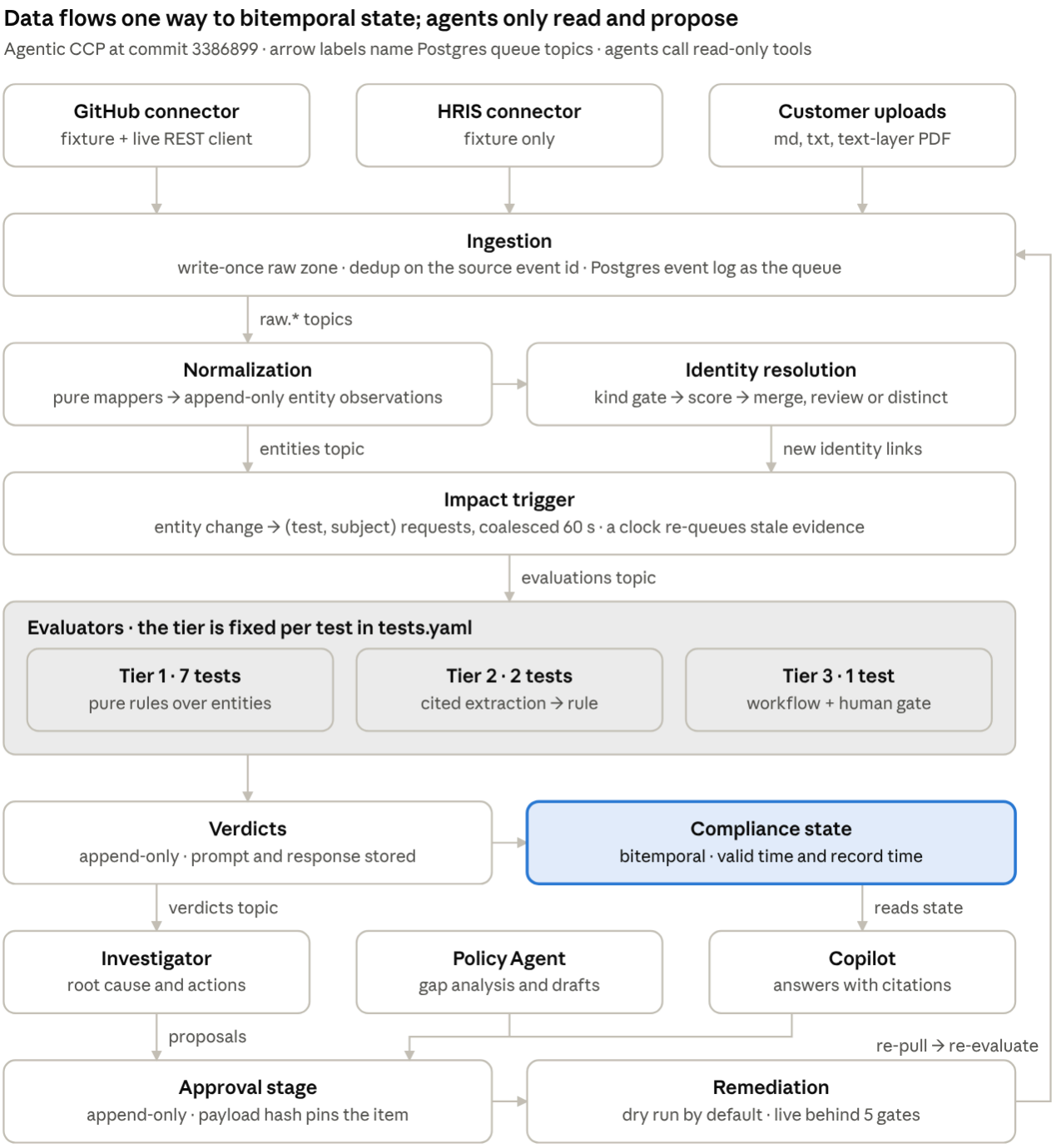
Evidence labels. Code facts carry a file and line, read at `3386899` (Sources). Competitor facts in B are marked Verified (stated on a page we opened), Inferred (our reasoning) or Claim (marketing we could not check). Later sections link each competitor fact to its source or to Section B, and mark our reasoning as "inferred" or "in our reading". Anything we could not check in code or on a page, such as how GitHub behaves when collaborators change, is marked unverified. Every effort figure is our estimate, in engineer-weeks. GH-1 to GH-5 and HR-1 to HR-5 are CCP's ten tests (A.3), and ids such as A-17 are numbered assumptions in `spec.md` . D1–D8, E1–E5 and the Q, M and L roadmap items are this report's own labels.

A. Current Agentic CCP architecture

Agentic CCP (`agentic/ccp`) is a 21,515-line Python build of the whole compliance loop, written from `spec.md` v1.3 in 17 phases on 11–12 Sep 2026. Its data-integrity core is production-grade in design: an immutable raw zone, append-only entities, a bitemporal state table, and "models extract, code decides". Its integration layer is the thinnest part: three connectors, one live, and no credential, rate-limit or sync-completeness handling.

Scale today, verified from the repo at commit `3386899` :

Dimension	Agentic CCP
Tenants	2 fixture tenants (`acme` , `globex`) in `config/tenants.yaml`
Connectors	GitHub (fixture plus a live REST/GraphQL client), HRIS (fixture only), manual uploads (md, txt, text-layer PDF)
Tests	10: 7 deterministic, 2 tier 2 (GH-4, HR-3), 1 tier 3 (HR-4)
Controls and frameworks	9 controls mapped to a 6-criterion SOC 2 slice and an 8-control ISO 27001 slice
Agents	Tier-3 evaluator graph, Investigator, Policy Agent, Copilot (all LangGraph)
Code and tests	21,515 lines in `ccp/` ; 15 SQL migrations; 66 test files with 383 test functions
Interface	CLI only (`ccp/cli.py`); no API, UI or user identity



Agentic CCP · components and data flow

Labelled arrows are rows in the Postgres event log; unlabelled arrows are direct calls. The only path from an action back to state is the loop on the right: remediation re-pulls the source, and the new observation is evaluated like any other change.

A.1 Components

Layer	Code	What it does	Property that matters
Connectors	`ccp/connectors/base.py:26-54`, `registry.py:28-53`	`pull(tenant, since)` yields `SourceEvent`s carrying the provider's own id and timestamp, payload untouched	One contract for fixture and live sources
Ingestion	`ccp/ingest/landing.py:39-79`, `raw_zone.py:52-72`	Writes the raw file once, dedups on (tenant, connector, source event id), appends to `event_log`	The raw zone is a replayable audit record
Queue	`ccp/ingest/event_log.py:85-101`, `consumers.py:45-136`	Postgres table as a queue: ordered topics with per-tenant cursors; parallel topics with `SKIP LOCKED`, 5 attempts, dead-letter and parking	Model calls run outside transactions, except in the Investigator (D4)
Normalization	`ccp/normalize/consumer.py:53-88`, `*_mapper.py`	Pure mappers emit `Person`, `AccessGrant`, `Asset`, `Evidence`, `Vendor` observation rows	Append-only; "current" is the latest observation
Identity	`ccp/identity/`	Blocking, a kind gate (service and shared accounts never merge), weighted scoring, routing to merge, review or distinct	Every comparison is logged with its features and thresholds

Layer	Code	What it does	Property that matters
Knowledge	`ccp/knowledge/*.yaml`, `graph.py`, `rollup.py`	Framework → requirement → control → test, loaded as reference tables	One verdict backs requirements in both frameworks
Impact trigger	`ccp/evaluators/impact.py:149-209`	Maps each entity change to the (test, subject) pairs to re-run, coalesced for 60 s	Change-driven, not schedule-driven
Evaluators	`ccp/evaluators/runner.py`, `deterministic/`, `tier2.py`, `ccp/agent/tier3.py`	Budget gate, then tier 1, 2 or 3; one verdict per `evaluation_key`	An infrastructure failure parks or dead-letters; it never becomes a verdict
State	`ccp/storage/bitemporal.py:106-135`, `ccp/compliance_state/`	Bitemporal `compliance_state` with a per-subject advisory lock and a per-tenant sequence for transaction time	Answers "what was true" and "what did we believe" separately
Agents	`ccp/agent/core.py`, `tools.py`, `allowlists.py:19-30`, `graphs/`	Four LangGraph graphs on one read-only tool module	Graphs read, runners write
Approvals	`ccp/approvals/store.py`	One append-only table for interrupts, remediation, policy drafts and evidence requests	State is the latest event; a payload hash pins what was approved

Layer	Code	What it does	Property that matters
Remediation	`ccp/remediation/gates.py:53-84`, `ccp/evaluators/targets.py:55-115`	Dry run by default; one live action behind five gates; targets must be cited by the verdict	Re-evaluation closes through the same impact trigger
Observability	`ccp/observability/steps.py`, `ccp/replay/rebuild.py`, `ccp/evals/`	JSONL step log with GenAI attribute names, Langfuse spans, replay, golden, judge and adversarial evals	Every model verdict can be rebuilt byte for byte, until a cited document changes (A.7)

A.2 Integration lifecycle, end to end

There is no "customer connects an account" step: the GitHub token comes from `GITHUB_TOKEN` and tenants from a YAML file. From there:

1. **Pull.** `ccp ingest` calls `connector.pull(tenant)` with no cursor, so every run is a full pull (`landing.py:100`). The live GitHub client reads 7 REST endpoints and one GraphQL query (`github/client.py:1-13`).
2. **Land.** Each event is written under `data/raw/{tenant}/{connector}/{yyyy/mm/dd}/`, then inserted into `raw_events` with `ON CONFLICT DO NOTHING`; only a new row reaches `event_log` (`landing.py:50-77`).
3. **Normalize.** One ordered consumer per raw topic re-reads the raw file, runs the pure mapper, inserts observation rows and announces each new row on the `entities` topic (`normalize/consumer.py:66-85`).
4. **Resolve identity.** HR and GitHub persons are linked into a canonical person; ambiguous pairs go to a review queue and make the dependent tests `needs_review`.
5. **Trigger.** The impact consumer turns each entity change into evaluation requests, folding duplicates for the same subject inside 60 s (`impact.py:81-113`).
6. **Evaluate.** The evaluations consumer checks the tenant's daily budget, routes by tier and inserts the verdict idempotently on `evaluation_key` (`0005_state.sql:34`).
7. **Record state.** One `compliance_state` row per verdict supersedes the subject's chain head under an advisory lock (`bitemporal.py:125-135`). A `needs_review` verdict is written as a fact, not held back.

8. **Fan out.** Fails and `needs_review` verdicts go to the Investigator, whose proposals land in the approval stage. A clock producer re-enqueues subjects whose evidence passed `evidence_max_age` (`ccp/evaluators/clock.py`).
9. **Remediate and re-evaluate.** An approved proposal runs as a dry run by default; the re-pull flows back through steps 1–7, so a fixed issue shows as a new `pass` row superseding the `fail`.

A.3 Compliance evaluation: how a verdict is made

The tier is fixed at authoring time in `ccp/knowledge/tests.yaml`, never chosen at run time: `deterministic` goes to tier 1, `interpretive` with one evidence type to tier 2, and two or more types to tier 3.

Tier	Tests	What a model does	What decides	Confidence
1 Deterministic	GH-1, GH-2, GH-3, GH-5, HR-1, HR-2, HR-5	Nothing; `tests/unit/test_layering.py` fails if a tier-1 module imports `ccp.llm` or `langgraph`	A pure function over entities	1.0; `needs_review` only when an identity link is unresolved
2 Extract-then-rule	GH-4, HR-3	Fills a typed schema; each field cites a chunk id and a verbatim quote	A pure rule over verified fields (`ccp/evaluators/rules/`)	Field coverage × citation survival × retrieval percentile, with forcing caps
3 Fixed workflow	HR-4	Extracts per evidence type in a LangGraph workflow; a bounded tool loop runs only when a fetch finds nothing	A pure comparison, then a human gate below 0.70	As tier 2, plus a 0.40 cap for an unresolved evidence type

Evidence enters a prompt only through `prompt_guard.frame_evidence()`, which redacts secrets, wraps each chunk in a per-call random fence and caps confidence at 0.60 when it sees injection markers. After generation, `citations.py:87-129` drops any field whose quote is not found in the cited chunk. Only dates, providers and timeline text must also appear in their quote, so the fields that decide pass or fail accept any quote that is present (A.7). A verdict left without citations becomes `needs_review`.

A.4 Agent execution

Graph	Runs when	Tools (allowlist)	Produces	Persisted by
Tier 3 evaluator	An HR-4 evaluation	7 read tools; no prior verdicts, so it cannot anchor on them	A verdict, or an interrupt for review	The evaluations runner
Investigator	Every `fail` or `needs_review` verdict	Tier-3 set plus `get_verdict`, `list_findings`, `get_access_grants`, `get_control`	Root cause, missing evidence, actions from a closed enum	Its runner, one approval per action
Policy Agent	`ccp policy gap` or `draft`	`search_evidence` (policy documents only), `get_chunk`, `get_control`, `get_framework_rollup`	Gap report; Markdown draft under `data/drafts/`	Its runner; a draft becomes evidence only through a human upload
Copilot	A question in the REPL	All 13 read tools	Answer whose citations are checked for presence; at most 2 requested actions per turn	Its runner, after a typed `yes`, opens approvals

Every tool takes the tenant from a closure, rejects extra arguments (`tools.py:35-36`) and answers off-list calls with an error result (`core.py:340-341`). Runs are bounded by token, dollar, wall-clock and tool-call budgets, checkpointed to Postgres under row-level security, and resumable from a separate process.

A.5 Data, background work and failure handling

- **Append-only by grant.** The app role is granted only `SELECT` and `INSERT` on verdicts, compliance state, approvals, investigations, gap reports, policy drafts and Copilot answers, and likewise on the raw-event index and the entity tables, so it cannot rewrite history (migrations 0005 and 0011–0014; the `REVOKE` lines there only restate intent).

- **Entities are observations, not state.** Each row carries the source timestamp, and a view returns the latest per entity (`0003_entities.sql:100-102`). Only `compliance_state` is bitemporal (assumption A-7).
- **Each worker process runs one consumer and polls every 2 s** (`worker.py:67-103` , `cli.py:552-563`). Parallel topics make up to 5 attempts, then dead-letter; exhausting the tenant's daily budget, or a missing model key, parks a record instead of failing it.
- **Model errors are classified.** 429, 529 and timeouts back off and retry; 400s, schema refusals and per-run budget overruns dead-letter at once, so a model outage never produces a verdict. Retrieval degrades instead: an embedding or reranker outage caps confidence, and yields a `needs_review` verdict if lexical search also finds nothing.

A.6 Security, isolation and evaluation

Isolation is enforced three times: a typed `TenantContext` , a `ScopedDB` that sets `app.tenant_id` per transaction, and forced row-level security on every tenant table, including vectors and LangGraph checkpoints. Manual-upload file paths are the one gap (A.7). `spec.md` §15 lists 21 guardrails, each with the test that proves it. The CI workflow runs ruff, strict mypy, an import-layer contract, migrations, pytest and keyless golden evals, and a nightly keyed job fails on any pass-to-fail flip, attack-success-rate increase or p95 breach. Neither has run in CI: the repo has no git remote. All 27 recorded eval runs are local, git-ignored files (`.gitignore:4`) in which every case that needs a model was skipped because no API key was set, so they cost nothing. AI-tier quality is therefore unmeasured.

A.7 Where the design stops (verified limits)

- **No connection lifecycle.** There is no OAuth, stored credential, token refresh, connection health or per-connection schedule; `pull()` is the whole contract (`base.py:49-54`).
- **The live client is not hardened.** It has no retry, backoff or rate-limit handling; any non-404 error aborts the pull mid-stream (`client.py:55-60`), and a GraphQL failure silently returns no verified emails (`client.py:80-81`).
- **Revocations are invisible in live mode.** A grant becomes `revoked` only from a webhook-style removal event (`github_mapper.py:303,321`). The live client emits snapshots only, so a removed collaborator keeps its last `active` observation. The end-to-end remediation test passes only because a test-only helper injects the removal event.

- **Unchanged snapshots re-land on every pull.** Member, branch-protection, `SECURITY.md` and file-listing snapshot ids embed the pull time (`client.py:129`, `:179`, `:189`, `:222`), so each live pull adds new raw files and observation rows even when nothing changed.
- **Citation checks confirm presence, not support.** Only dates, providers and timeline text must appear in their quote; the fields that decide pass or fail accept any quote found in the chunk, and a 0.85 word-overlap fallback ignores order and negation (`citations.py:42-47`, `:112`). Run on the repo's own fixtures, a training record marked "In progress" verified as completed and passed; spec A-17 records part of this as a known gap. Tier 3 fixes retrieval quality at 1.0, so a system the model leaves out never lowers confidence (`agent/graphs/tier3.py:189`).
- **Approvals can fail open.** `ccp approvals approve` does not validate `--decision`, so `reject` or a typo approves the draft (`approvals/cli.py:88-90`, `graphs/tier3.py:231-248`). No resume path checks the approved payload hash against the checkpoint, and `ccp remediate rollback` re-grants access without the five gates (`remediation/runner.py:243-289`).
- **Prompt-injection handling only detects.** Tool results in all four agent loops reach the model unfenced, and only the Copilot scans them (`llm/client.py:126-140`, `graphs/copilot.py:350-357`). Detection is 15 fixed English phrases, and a tier-2 hit only caps confidence (`prompt_guard.py:28-45`). Code still decides every status, and every action needs approval.
- **Secrets are redacted only in prompts.** Chunk text goes unredacted to the embedding and rerank APIs, the six secret patterns miss formats such as AWS secret keys, and GH-5 passes a repository whose files were not all read (`retrieval/index.py:139-151`, `secrets_patterns.py:11-20`, `gh5.py:51-54`).
- **Upload paths are not contained.** A manifest entry such as `../globex/report.pdf` is read and stamped with the wrong tenant (`manual/uploads.py:88`). Only the operator writes manifests today, so no tenant can exploit it yet, but it must be closed before customers can upload (D5).
- **A verdict can be lost between two transactions.** It is committed, then announced in a second transaction; a crash in between means it never reaches compliance state or the Investigator, and nothing reconciles them (`evaluators/runner.py:93-106`).
- **One bad record stalls an ordered topic.** An exception inside `OrderedConsumer.handle()` rolls back the whole batch and the cursor never advances, so the same record fails on every run and that tenant's topic stops (`consumers.py:61-72`). Only parallel topics dead-letter. The Investigator is an ordered consumer too, so an exhausted budget or a model error stops it the same way.

- **The model budget parks deterministic tests.** One daily budget per tenant is checked before the tier is known, so after $5 of spend even tier-1 tests park, and no shipped command un-parks them (`evaluators/runner.py:151-153`, `:231-236`).
- **The change-driven path skips identity resolution and indexing.** Only full runs resolve identities and refresh the index (`compliance_state/run.py:65-67`), so a GitHub account first seen after the last full run stays unlinked, and GH-1 and HR-1 can pass for a terminated employee until the next one.
- **Replay breaks once a cited document changes.** Replay looks chunks up in the live index, which drops superseded chunks, so earlier tier-2 and tier-3 verdicts can no longer be replayed, although the stored request still holds the fenced text (`replay/rebuild.py:168-173`). Verdicts also store no rule version.
- **Safe only as one process.** With more than one consumer, ordered cursors can skip rows that commit out of order, and a stale verdict can replace a fresh one (`consumers.py:62-71`, `writer.py:67-77`). The shipped single-process flow avoids both.
- **The impact table is code.** Test ids are hard-coded in `requests_for_entity` (`impact.py:149-209`), although `spec.md` Layer 7 says the mapping is a table. A test left out of it is silently never triggered by changes.
- **Teaching scale by design.** Worker processes that each poll one consumer, a local-disk raw zone, no API or UI, and typed reviewer names instead of authenticated identities.

B. Competitor technical research

The six incumbents keep a current-state store, fed mostly by scheduled syncs (Sprinto also accepts webhooks), and run deterministic tests over it. Audit history differs: run logs or snapshots (Vanta, Drata, Scrut), evidence bound to each audit (Thoropass), or no integration-test history at all (Secureframe). LLMs work around the pass/fail decision, not inside it. None publicly documents an immutable event ledger, bitemporal state or citations verified in code; the closest is Sprinto's automatic grounding check, whose mechanism is not documented. The AI-native entrants differ mainly in where their agents sit and how much they may write. In 2026 the Delve allegations made evidence provenance the category's credibility test.

Labels: **Verified** means stated in a primary source our research opened on 9 Nov 2026 (linked). **Inferred** is our reading of those facts. **Claim** is vendor marketing we could not check. Internal implementations are private; nothing below states one as fact unless the vendor published it.

B.1 Vanta

By its own count the largest catalogue (400+ integrations), and in our reading the most developed agent platform, on a pooled current-state store.

- **Verified:** a pushed integration replaces a resource type in full on every sync, and resources left out are deleted (Build Integrations overview). The push API accepts 11 built-in resource types, plus a flat custom type that only private integrations may use; native integrations have resource kinds of their own (Resources).
- **Verified:** for synced resources, tests read Vanta's stored copy, and API-introspection tests also record the requests made during the run. The developer docs say pushed data syncs typically hourly and that tests re-run on the next sync, "usually within 24 hours"; marketing says hourly (Tests, Remediate with MCP).
- **Verified:** audit history is pass/fail timelines plus test-snapshot rows holding raw JSON and, for API-introspection tests, the requests made during the run (test snapshot API).
- **Verified:** custom tests are no-code, join at most two sources on one field, cannot be edited after saving, and have no API or MCP path.
- **Verified:** the Agent and the hosted MCP server act with the user's permissions; the developer docs say MCP has no read-only mode, though the MCP FAQ says write actions need re-authorization with write permissions. Approval steps exist per feature: custom agents run autonomously with their creator's permissions and stop at "Needs approval" only for suggested changes, such as policy or SLA updates. Since May 2026 the offboarding workflow deprovisions Zoom and Figma users, and the MCP remediation flow asks before running CLI changes against live resources (what's new, Remediate with MCP).
- **Verified:** AI quality rests on verifiers. An LLM evidence evaluator passes or flags evidence; Vanta measures it on an expert-labelled dataset, with LLM-judge metrics, and by agreement with auditors' accept-or-flag decisions. No citation-checking mechanism is documented.
- **Verified:** MongoDB Atlas is the primary database (blog). Vanta's RCA for Inc-868 (incident 22–27 May 2025) says a code change removed a "domain ID filter", so data pushed through third-party integrations landed in other customers' tenants (trust updates).
- **Verified:** AI entries on the subprocessor list include E2B sandboxes, Hatchet workflows, kernel.sh, Reducto and Lakera, with models from OpenAI, Anthropic, Google and Microsoft. Merge collects data for some HRIS, task-tracker and access integrations.

- **Inferred:** native connectors are scheduled pulls whose internals are unpublished. A job posting sets the integrations platform's goal as "declarative, low-code, and no-code patterns, supercharged by AI agents".

B.2 Drata

Daily deterministic tests, a JSON rule language for custom tests, and agents concentrated in vendor risk and trust.

- **Verified:** enabled tests "run daily with Autopilot", and each run is stored as a test event with raw JSON (Monitoring, Test details).
- **Verified:** completing a custom-connection upload session "atomically replaces the active dataset" and permanently deletes the previous records.
- **Verified:** custom tests are JSON rules (`all` / `any` , `fact` / `operator` / `value` / `path`) over one resource type per condition group, capped at 70 lines per block (Advanced editor). **Inferred:** the schema matches the open-source json-rules-engine.
- **Verified:** no documented feature lets a model decide pass or fail (our reading of the docs). AI summaries of failures are labelled "assistive, not authoritative".
- **Verified:** the MCP server's September 2026 tool table lists 36 tools, 14 of them writes; its OAuth server metadata also advertises scopes with no documented tool, such as re-running tests and excluding findings. Access is the intersection of granted scopes and the user's role. The TPRM Agent shows the source excerpt behind each criterion and asks before each write; per-checkpoint autonomy settings can let it proceed without pausing, including when recording risk decisions, though the default still asks first.
- **Verified:** HRIS data arrives through Merge's Common Model (Workday guide). The full subprocessor list (51 entries) names OpenAI for SafeBase AI features such as questionnaire answering, Anthropic for AI Agent Governance, and AI vendors including Vellum, Reducto and Browserbase, plus Braintrust, which Drata says it uses only for internal, offline evaluation.
- **Verified:** a job posting (removed 23 Sep 2026) listed Temporal, Node.js, TypeORM and MySQL as required skills, with NestJS preferred. A 2025 post describes a single-tenant database architecture in which each customer gets its own database partition.
- **Verified:** by default auditors see only sampled evidence inside the audit window; a customer can also grant an auditor read-only access to the whole application.

B.3 Secureframe

Breadth through aggregators, latest-sync-only test state, and a write-capable MCP server.

- **Verified:** "Integration tests only evaluate data from the most recent sync", and Secureframe "does not keep historical integration test data in the app" (Managing connections). Syncs run daily, and the page shows connection and sync status with the raw error.
- **Verified:** integration subprocessors include Finch, Leen, WorkOS and OneSchema (subprocessors). Custom integrations accept API or CSV data with customer-defined schemas.
- **Verified:** Custom Automated Tests are no-code conditions that run only once mapped to a control. Changes to built-in tests ship as versioned updates that are added automatically after a countdown (14 days by default) or by a Secureframe-set deadline at least three weeks out.
- **Verified:** Comply AI remediation produces copy-paste IaC and CLI fixes, and AI evidence validation is advisory only.
- **Verified:** the hosted MCP server exposes 100+ tools with create, update and delete, has no read-only mode; Secureframe advises reviewing each action and connecting as a restricted role. Secureframe also holds FedRAMP 20x Moderate.

B.4 Thoropass

A platform built around its own audit firm, with AI at evidence intake and on the auditor's side.

- **Verified:** audits come from Thoropass's own CPA firm and, since September 2026, its own accredited ISO 27001 certification body (platform, newsroom).
- **Verified:** evidence requests belong to one audit, and "evidence from previous audits can never be used for current audits" (help). Connected documents cannot be refreshed while their request is submitted to the auditor.
- **Verified:** Smart Sort maps uploaded files, including exports from Vanta and Drata, to evidence requests. An open PM role says its LLMs "draft first-pass test results" for auditor review.
- **Verified:** the MCP server is documented as writing only comments, attachments and status updates on evidence requests, though its OAuth metadata also lists `globalControl:write` and `training:write` scopes, and its MCP post says "Thoropass will remain the system of record".
- **Inferred:** integrity comes from freezing evidence per audit, not from continuously versioned state. No customer-built monitors are documented. The help-center release notes end in July 2025, though the newsroom lists 2026 launches such as Smart Sort and MCP.

B.5 Sprinto

The closest public match to a generic, data-driven ingestion engine.

- **Verified:** ingestion "plans" are written in AI, visual or YAML editors (get started). A typical webhook plan runs transform → map → insert → evaluate into customer-definable entity types that upsert on an identifier and carry `lastSeenOnSource` (ingestion guide, entity types). No raw-event store is documented: "A 202 confirms receipt, not storage."
- **Verified:** Merge (stores normalized data), Truto (pass-through), Leen (security data) and Kombo (LMS) serve as aggregators (aggregators, subprocessors). Most integration checks refresh every 4 to 12 hours (FAQ).
- **Verified:** the Entities Sync API is session-based: start, send pages, then apply or discard.
- **Verified:** the Fix-it extension makes changes in customer consoles through the user's logged-in browser after confirmation, and MCP writes follow the user's role.
- **Verified:** the August 2026 questionnaire pipeline separates fact extraction from answer writing and adds a grounding check with per-answer sources.

B.6 Scrut

Direct connectors, vendor-authored tests, and narrowly scoped agents.

- **Verified:** documented connectors use direct API keys with 24-hour syncs (for example 15Five); scheduled tests run every 24 hours and Continuous Monitoring tests in real time; no aggregator appears among the subprocessors. Tests are generated by Scrut from a CIS-based library, with no customer test builder documented (Tests walkthrough).
- **Verified:** MCP offers 19 read tools and 3 writes (evidence upload and Trust Vault access decisions). The REST Open API requires an `Idempotency-Key` on every write.
- **Verified:** Teammates agents propose rather than act, for example IaC fixes with a confidence score. A vulnerability Validation Agent reproduces each finding before tagging it Likely TP or Likely FP.
- **Inferred:** "risk-first" means every module can raise risks into the register. Nothing shows risk deciding which controls are in scope or which tests run.

B.7 AI-native and agent-first entrants

Company	What the agents do	Validation and autonomy	Notable
Delve	Screenshots, reports, evidence validation	Acts	TechCrunch reported allegations of fabricated evidence and pre-written auditor conclusions; Delve denies fabricating evidence. On 4 Apr 2026 TechCrunch reported that it had parted ways with YC; YC gave no reason.
Comp AI	Policy drafts, browser evidence capture, a check DSL stored as data (public repo, Jul–Aug 2026)	A person approves agent-drafted policies; the founders say approval should grow as agents take on more consequential actions	$34M Series A (Sep 2026). The open repo shipped "P0 auth and tenant-scoping" fixes on 23 Sep 2026 and became a "Community Edition" on 22 Oct 2026.
Probo	Up to 16 sub-agents for vendor vetting; 270+ MCP tools per its README	Agents read and write; document approvals exist	MIT, Go. Its vetting save tools store vendor document URLs but have no field linking a fact to its source.
Anecdotes	A data engine with 230+ plugins, plus a no-code Agent Studio	Not documented	Collects structured system data first, then runs agents on it: in our reading, the layering closest to Agentic CCP.
Zania	Evidence, control testing, questionnaires	Citations and confidence (claimed)	$18M Series A led by NEA.
Complyance	16 live agents, including TPRM	Flags risks for human review	$20M Series A led by GV.

Company	What the agents do	Validation and autonomy	Notable
Fieldguide	Agents inside audit-firm engagement teams	Human oversight	Says half of the top 100 US accounting firms use it. Our inference: auditor-side AI may shape which evidence formats auditors accept.

Across these entrants the recurring pattern is agents on document-heavy work (questionnaires, vendor review, policies) and deterministic check catalogues for cloud configuration. Several ship MCP; Probo documents read and write access. Vendors such as Zania claim citations, confidence scores and human sign-off, but none publishes an evaluation method or a citation-verification step.

B.8 Standards direction

- **Verified:** FedRAMP 20x's 2026 certification rules require machine-readable JSON valid against FedRAMP's own JSON schemas. They do not mention OSCAL, and FedRAMP's proposed Rev5 RFC-0024 lists OSCAL as one approved format. They also set a minimum number of automated validation methods per Key Security Indicator: at least 2 for Class C and at least 4 for Class D. FedRAMP stops accepting new Rev5 certifications on 11 Jun 2027.
- **Verified:** Lula 2 dropped OSCAL as its working format because it "proved too complex for most teams", while OSCAL 1.2 added a control-mapping model.
- **Inferred:** keep a simple internal model in which each result records its validation method, observed and recorded times, and hashed evidence links. Export OSCAL, FedRAMP JSON or OCSF at the boundary.

C. Competitive architecture comparison

In design, CCP leads the documented competitors on evidence history and on limits for what agents may do, but both leads have defects to fix first (D3, D5, D6), and no AI tier has been measured on a real model (D7). It trails the incumbents on connectors and product surface, and most of them on customer extensibility (D1, D8). No compliance vendor documents citation checks in code, and no incumbent documents what a partial sync from a native connector does, so checked AI output (D2) and complete-sync rules (D1) are where CCP can pull ahead. The CCP column was verified in code at `3386899` (paths

under `ccp/`, migrations by file name) and the competitor column on the linked pages, most of them cited in Section B; the last column is our assessment, then our recommendation.

Capability	Agentic CCP today	Competitor approach	Gap / opportunity for CCP
Integration architecture	One `pull()` contract (`connectors/base.py:49-54`) and three connectors: GitHub (fixtures or a live client), HRIS (fixtures only) and manual uploads (`connectors/registry.py:28-53`). No OAuth, stored credentials, refresh or health; the token comes from `GITHUB_TOKEN` (`registry.py:31-35`). The live client has no retry or rate-limit handling, and any error but a 404 aborts the pull (`connectors/github/client.py:55-60`)	Native connectors plus aggregators: Merge at Vanta (help); Finch and Leen at Secureframe (subprocessors); Merge, Truto and Leen at Sprinto (aggregators), which also builds some connectors, such as Wiz, as ingestion plans (Wiz) written in AI, visual or YAML editors (ingestion). Comp AI keeps its long tail of checks as versioned data, in its own JSON check language (repo)	Far behind (D1). Before adding connectors, build what breadth depends on: a connect step (OAuth or a cross-account role) with stored, refreshable credentials and a health status; cursors, so a pull resumes where the last one stopped; a record of whether each pull was complete; and a test suite every connector must pass. Then grow breadth with checks stored as data and AI-drafted connectors that the suite and a reviewer gate
Customer extensibility	None. There is no API, and data enters only through the three connectors (`connectors/registry.py:16`)	Push APIs with customer-defined schemas: Vanta private integrations (resources), Drata custom connections (recipe), Secureframe custom integrations (blog) and Sprinto entity types (docs). Scrut accepts pushed evidence and scanner findings only (Sep 2026)	Behind (D1, D8). Add a push API whose sessions land in the raw zone like any connector, with customer-defined resource schemas. It needs the completeness record and an authenticated API first
Sync semantics	A full pull on every run (`ingest/landing.py:100`). The live client emits snapshots only (`connectors/github/client.py:1-13`), but only a removal event revokes a grant (`normalize/github_mapper.py:303`, `:321`), so a collaborator removed upstream stays active. Member, branch-protection, `SECURITY.md` and file-listing snapshot ids embed the pull time, so unchanged data lands again on every pull (`client.py:117-130`, `:177-231`)	On push APIs: each sync replaces a resource type in full and deletes what it left out (Vanta); a finished session atomically replaces the dataset (Drata); a session is applied or discarded (Sprinto). No incumbent documents what a partial sync from a native connector does. Comp AI's code resolves findings only from successful scans and carries unscanned services forward as unknown (code)	Behind (D1). Record whether each pull was complete. Only a complete pull may turn an absence into a revocation, written as a new observation, never a delete; after a partial pull, what was not seen stays unknown. Derive snapshot ids from content, so unchanged data lands once

Capability	Agentic CCP today	Competitor approach	Gap / opportunity for CCP
Evidence and history	Write-once raw files (`ingest/raw_zone.py:52-72`); verdicts and state granted only `SELECT` and `INSERT` (`0005_state.sql:66`); bitemporal state (`storage/bitemporal.py:106-155`)	Run logs and test snapshots (Vanta, Drata); the latest sync only (Secureframe); evidence bound to one audit (Thoropass)	Ahead in design, unproven at scale; demand for as-of history is untested (inferred). Store a rule version on each verdict, and keep cited text replayable after a document changes (D6). Move the raw zone from local disk to object storage with a retention lock (D8)
Monitoring cadence	Pulls run only when an operator runs a command such as `ccp ingest run` or `ccp run` (`cli.py:80-97` , `:529-549`); no worker pulls on a schedule. Evaluation is change-driven, with repeats folded together over 60 s (`evaluators/impact.py:81-113`), plus a clock for stale evidence (`evaluators/clock.py:1-7`). Identity resolution and index refresh run in a full run or by hand, never on the change-driven path (`compliance_state/run.py:66-67`)	Daily (Drata, Secureframe); every 24 h, with some tests in real time (Scrut); every 4–12 h (Sprinto). Vanta re-runs tests on the next sync, "usually within 24 hours", and pushed data typically syncs hourly (remediate, push)	Ahead on what triggers evaluation, behind on collection. Change-driven evaluation re-runs only what changed, but a result is only as fresh as the last pull. Add a schedule per connection (D1), and resolve identities and refresh the index on every change, not only in full runs (D6)
Built-in test logic	Pure rules over typed entities, one module per test, with no model call (`evaluators/deterministic/__init__.py:1-25`). The tier is fixed when the test is written (`knowledge/graph.py:111-115`). Verdicts store the model and prompt version but no rule version (`0005_state.sql:6-33`)	Tests read stored resources, not the source system, and built-in tests are versioned by the vendor (Vanta). Built-in test changes arrive as versioned updates after a countdown (Secureframe). An offline AI pipeline wrote 1,000+ deterministic tests, all reviewed by auditors (Drata)	At par. Store the rule version on each verdict, and release rule changes with notice, as Secureframe does, plus a preview of the verdicts they would change (D6). Let AI draft new checks offline, gated by tests and a reviewer, so verdicts stay deterministic
Test verdicts	`pass` , `fail` , `partial` or `needs_review` (`0005_state.sql:12`). A model outage parks, retries or dead-letters the request; it never becomes a verdict (`llm/errors.py:1-5` , `ingest/consumers.py:98-125`). An embedding or reranker outage lowers confidence instead (`evaluators/confidence.py:13`). A requirement takes the worst state of its tests, and a tenant with no results reads as "no evaluations yet", never as clean (`knowledge/rollup.py:1-9`)	Pass or fail per resource, rolled up to each mapped control (Vanta); AI summaries of failures are "assistive, not authoritative" (Drata)	At par. Keep the four outcomes, and record each verdict's method and rule version (D6). Once pulls record completeness, return `needs_review` for a test that rests on an incomplete pull (D1)
Customer-defined tests	None. A new test needs a YAML entry (`knowledge/tests.yaml:1-3`), a rule module (`evaluators/deterministic/__init__.py:17-25`) and a hand-written branch in the impact table (`evaluators/impact.py:149-209`)	No-code rules: Vanta joins at most two data sources on one field (custom tests); Drata combines condition groups, each over one resource type (editor); Secureframe (tests) and Sprinto (checks) use field conditions, with no joins documented. Scrut and Thoropass document no customer test builder	Behind. First give CCP's own rules versions and move its impact table into data (D6). Offer a customer builder only after pulls record completeness (D1), because rules over incomplete data give wrong answers faster

Capability	Agentic CCP today	Competitor approach	Gap / opportunity for CCP
Framework mapping	9 controls in YAML, mapped to a 6-criterion SOC 2 slice and an 8-control ISO 27001 slice (`knowledge/controls.yaml:1-10` , `knowledge/frameworks/soc2.yaml:6-51` , `iso27001.yaml:6-68`). One test result backs both frameworks, and a requirement takes the worst state of its tests (`knowledge/rollup.py:1-9`)	Common-control libraries: Vanta's, in preview, maps supported frameworks automatically and applies a suggested mapping only once a user confirms it (common controls); Drata's mapping agent proposes controls from its library with a confidence indicator and adds nothing until a person accepts (mapping agent)	Far behind on breadth. Grow to a versioned common-control catalogue. Let an agent propose mappings that cite the requirement and control text, and have a reviewer accept each one: a wrong mapping changes every requirement status built on it
People and identity	Blocking, a gate that never merges a person with a service or shared account, weighted scoring, a review queue and a log of every decision (`identity/__init__.py:1` , `identity/scoring.py:1-13`). It runs in a full run or by hand, never on the change-driven path (`compliance_state/run.py:66` , `cli.py:458-465`), and a reviewer closes an item by typing a name (`cli.py:473-492`)	HRIS records matched to identity-provider records by email, then by name (Drata); AI-suggested matches that a person approves or dismisses (Sprinto)	Ahead in design. Resolve identities on every change (D6). Add an identity-provider connector keyed by immutable directory ids (D1), and a review screen for signed-in reviewers (D8)
Access reviews and remediation	The Investigator proposes actions for every `fail` or `needs_review` verdict (`investigation/runner.py:1-5`). One live action, removing a GitHub collaborator, runs only behind five gates (`remediation/gates.py:1-9`), but rollback re-grants access without them (`remediation/runner.py:243-289`). An access review is only a document, from the HRIS feed or an upload, that HR-4 reads (`knowledge/tests.yaml:98-102`)	Denying access in a review does not revoke it (Vanta, Drata, Scrut). Vanta deprovisions Zoom and Figma users on offboarding, and its MCP remediation flow asks before CLI fixes on live resources (what's new, remediate). Sprinto's extension changes customer consoles after the user confirms (Fix-it)	Safest design, least shipped (D3). Put rollback behind the same gates and fix the approval defects. Then differentiate on approvals bound to the exact payload, signed-in reviewers and receipts (records of each action and its approval), starting with changes a person applies, such as pull requests
Risk and vendor	Not built. The only vendor data is one `Vendor` record each for GitHub and the HRIS (`normalize/entities.py:107-114` , `normalize/github_mapper.py:147-157` , `normalize/hris_mapper.py:118-125`)	Agentic vendor review: Drata's TPRM Agent shows the source excerpt behind each criterion and, by default, asks before each write (TPRM Agent); Vanta's TPRM Agent collects vendor evidence and summarizes risks (Vanta); Probo runs up to 16 vetting sub-agents, whose save tools have no field linking a fact to its source (Probo)	Not built; room to lead on checked AI. Vendor review fits extract-then-rule: a model extracts cited fields from SOC 2 reports and questionnaires, code checks the citations (D2), and the risk owner decides

Capability	Agentic CCP today	Competitor approach	Gap / opportunity for CCP
AI output validation	Every cited field is checked in code (`evaluators/citations.py:87-129`), but for presence, not support: a 0.85 word-overlap fallback ignores order and negation (`:36-47`), and only dates, providers and timeline text must appear in their quote (`evaluators/extraction/__init__.py:32-57`). Confidence is computed, not reported by the model (`evaluators/confidence.py:1-15`), but tier 3 fixes retrieval quality at 1.0 (`agent/graphs/tier3.py:189`)	An evidence verifier scored against auditors' decisions (Vanta); a grounding check with per-answer sources (Sprinto); optional auto-approval of "high confidence" questionnaire answers (Drata's SafeBase). None documents citation checks in code	A lead once the checks test support (D2). Require a contiguous quote that contains the deciding value, check every field that decides the result, and measure retrieval quality in tier 3. Then require checked citations on every AI output, with accuracy measured on labelled cases (D7)
Agent autonomy	Graphs read, runners write: 13 read-only tools with the tenant bound in code, and one allowlist per graph (`agent/tools.py:1-8`, `agent/allowlists.py:19-30`). Approvals are append-only and pinned by a payload hash (`approvals/store.py:1-2`, `:22-24`), but an unknown decision approves (`approvals/cli.py:88-90`, `agent/graphs/tier3.py:231-248`), and tool results reach the model unfenced (`llm/client.py:126-140`)	Agents act with the user's permissions (Vanta, Drata, Secureframe, Sprinto); writes are narrow at Thoropass and Scrut. In Apr 2026 Vanta donated its AARM spec, which defines approval services and tamper-evident receipts for agent actions, to the Cloud Security Alliance	Strongest contract on paper; defects to fix (D3, D5). Validate the decision on the server, check the payload hash on resume, require signed-in reviewers, and fence and scan tool results. Then add a receipt for every agent action, the direction AARM sets
Agent interface	In-process tools only; no API or MCP server (`agent/tools.py:1-8`). The Copilot is a terminal REPL in which "fix this" ends in an open approval (`copilot/__init__.py:1-2`)	Hosted MCP, from 3 write tools (Scrut) or documented writes only on evidence requests (Thoropass) to 100+ tools with create, update and delete and no read-only mode (Secureframe)	Behind (D8). Generate an API and an MCP server from one typed tool registry, read-only first, with the tenant taken from the signed-in session. Writes arrive only as approval requests
Questionnaires and trust	Not built. The nearest pieces are tier-2 extract-then-rule and the Copilot, whose answer citations are checked for presence only (`agent/graphs/copilot.py:364-383`)	AI drafts with per-answer sources and optional auto-approval of "high confidence" answers (Drata's SafeBase); fact extraction kept apart from answer writing, plus a grounding check (Sprinto)	Not built; room to lead on answers with checked citations. Build on extract-then-rule with support checks (D2), reuse approved answers, and have a person approve what is sent. No auto-approval until accuracy is measured (D7)
Auditor view	`state_as_of(valid_at, known_at)` answers both audit questions (`storage/bitemporal.py:174-179`), one subject at a time from the command line (`cli.py:143-166`). No export or auditor access	Sampled evidence inside the audit window by default (Drata); test snapshots through an Auditor API (Vanta); evidence frozen per audit (Thoropass)	Ahead in the data model, with no surface. Export hashed, as-of audit packages that record each result's method and rule version (D6), and give auditors read-only access once user accounts exist (D8)

Capability	Agentic CCP today	Competitor approach	Gap / opportunity for CCP
Tenant isolation	Three layers: a typed tenant context (`tenancy.py:1-7`), `app.tenant_id` set per transaction (`storage/scoped.py:40`), and forced row-level security on every tenant table (`0002_rls.sql:16-19`), vectors and agent checkpoints included (`0008_chunks.sql:56`, `storage/checkpoints.py:54`). Agent tools read through the application role, which can also insert (`storage/db.py:34-35`, `0005_state.sql:66`). Upload paths are not contained (`connectors/manual/uploads.py:88`)	A database partition per customer (Drata); database-level safeguards and isolation tests, added after a 2025 incident put pushed data in other customers' tenants (Vanta)	Ahead in design. Contain upload paths to the tenant's own folder (D5), and give agent tools a read-only database role, so a tool bug cannot write (D8)
Background processing	A Postgres table as a queue: ordered topics with cursors, and parallel topics with `SKIP LOCKED`, 5 attempts, dead-letter and parking (`ingest/event_log.py:85-101`, `ingest/consumers.py:75-136`). The worker runs one consumer per process and polls every 2 s (`cli.py:552-563`, `ingest/worker.py:67-103`). One bad record stalls an ordered topic (`consumers.py:61-72`), a verdict is announced in a second transaction (`evaluators/runner.py:93-106`), and the daily model budget is checked before the tier is known (`:153`)	Temporal in a since-removed Drata job posting (posting); Hatchet on Vanta's subprocessor list, use undocumented (subprocessors); a Vanta posting asks for durable agent execution with "checkpointing, retries, human-in-the-loop steps" (posting)	Behind on durability and scale (D4). Commit each verdict and its announcement together, give ordered topics a dead-letter path, keep deterministic tests out of the model budget, and make consumers safe to run side by side. Then choose between hardening this queue and a durable workflow engine
Observability and evals	A JSONL step log with GenAI attribute names, mirrored to Langfuse (`observability/steps.py:1-8`), and replay of every model-backed record (`replay/rebuild.py:1-10`), which breaks once a cited chunk leaves the index (`:168-173`). Golden, judge and adversarial suites exist (`config.py:58-62`), but all 27 eval runs saved locally in `evals/runs/` (not committed) skipped every case that needs a model, because no API key was set. CI has never run: the repo has no git remote	An evidence verifier scored on an expert-labelled set and against auditors' decisions (Vanta); Braintrust for internal, offline evaluation only (Drata); a 30-question faithfulness benchmark scored 0.995 (Sprinto)	Behind until run (D7). Push to a remote so CI runs, run the suites on real models, replace the nightly gate that blocks on any single flip (`evals/regress.py:1-5`) with confidence bounds, and move the judge off the tier-2 model (`config.py:73-75`). Fix replay for changed documents (D6). Published results would be rare in the category (inferred)
Standards export	None. Verdicts store the tier, model, prompt version and evidence references, but no rule version (`0005_state.sql:15-30`)	FedRAMP 20x requires machine-readable JSON and a minimum number of automated validation methods per Key Security Indicator (rules); Vanta's Government Cloud (trust updates) and Secureframe (blog) hold 20x Moderate	Not needed until a regulated buyer asks. Then export each result with its method, rule version and evidence hashes (D6) to FedRAMP JSON or OSCAL at the boundary, keeping the internal model simple (B.8)

D. Architecture gaps and technical debt

CCP's core design (A.1–A.5) is sound, but the code around it can still record a wrong verdict, lose one, or run an action nobody approved, and CCP cannot yet be measured, fully replayed or run for a customer. Each finding gives today's code, verified at `agentic 3386899` unless marked otherwise, then what competitors do (with links), the gap, our

recommendation, how to build it, the tradeoffs and a priority. P0 means fix before CCP handles any real customer data or relies on a real model's output, P1 means needed for a first production customer, P2 means later, and every effort is our estimate in engineer-weeks.

#	Gap	Priority	Effort
D1	Live evidence collection: the connector contract has no lifecycle, cursor or completeness signal	P0 contract, P1 lifecycle and harness	3–4 wk, then 6–8 wk
D2	Citation checks confirm presence, not support	P0	3–4 wk, plus 1–2 wk of expert labelling
D3	Approvals can fail open	P0 fixes, P1 signed-in reviewers	1.5–2 wk, then 1–2 wk
D4	Background work is safe only as one process	P0 single-process defects, P1 several workers	2–3 wk, then 2–3 wk
D5	Untrusted content and secrets	P0 containment, P1 structural defenses	2–3 wk, then 2–3 wk
D6	Replay and change propagation	P0 identity on the change path, P1 the rest	1–2 wk, then 4–5 wk
D7	AI quality is unmeasured	P0	3–4 wk, plus labelling shared with D2
D8	No product surface or operating model	P1; P2 for MCP	12–16 wk

D1. Live evidence collection: the connector contract (P0, then P1)

- **Today.**
  - The contract is one method, `pull(tenant, since)` (`ccp/connectors/base.py:49-54`). There is no OAuth, stored credential, token refresh, connection health or schedule. The live GitHub connector takes a single token from the environment (`ccp/config.py:101`, `ccp/connectors/registry.py:33-35`).
  - Ingest never passes `since`, so every run is a full pull (`ccp/ingest/landing.py:100`). Each event commits on its own (`landing.py:45-78`), and connectors run one after another with no error handling

(`landing.py:92-102`). A pull that fails halfway keeps what it landed and skips the connectors after it. Nothing in the database marks the run as incomplete; only the step log records the error.

  - Revocations are invisible in live mode. Member and collaborator snapshots always map to `active` grants (`ccp/normalize/github_mapper.py:173-177`, `:207-218`). Only webhook-style removal events produce `revoked` (`github_mapper.py:303`, `:321`), and the live client emits snapshots only (`ccp/connectors/github/client.py:3-6`). A removed collaborator keeps its last `active` observation, so HR-1 and GH-1 cannot return to pass after access is removed, and the remediation loop never closes. The end-to-end test closes it only because a test-only helper injects the removal event (`ccp/connectors/github/fixture.py:157-167`, `:187-205`; `tests/e2e/test_remediation_loop.py:82-86`).
  - Unchanged snapshots re-land on every pull. The fixture reader keys snapshots by the provider's `updated_at` "so an unchanged snapshot dedups to nothing" (`fixture.py:3-5`), but the live client keys member, branch-protection, `SECURITY.md` and file-listing snapshots by the pull time (`client.py:129`, `:179`, `:189`, `:222`). Dedup compares ids, never the stored payload hash (`landing.py:52-65`). The repository snapshot, which carries the collaborator list, is keyed by the repository's `updated_at` (`client.py:135`, `:155`); if GitHub leaves that field alone when collaborators change (unverified), a changed list is dropped as a duplicate.
  - The live client is not hardened. It has no retry, backoff or rate-limit handling, and any error other than 404 aborts the pull (`client.py:55-60`). A failed GraphQL call returns no verified emails (`client.py:80-81`). An exact email match is worth 0.55 of the 0.85 score needed to link two records automatically (`config/identity.yaml:6`, `:15`), so one failed call can leave a GitHub account unlinked, where HR-1 and GH-1 cannot see it.

- **Competitors.** The incumbents document completeness only for their push APIs. Vanta replaces a pushed resource type in full on every sync and deletes what was left out (Build Integrations overview). Drata's custom-connection sessions "atomically replace" the dataset and permanently delete the previous records (custom connections). Sprinto's sync sessions are applied or discarded (Entities Sync API), and its entity types carry `lastSeenOnSource` (entity types). Comp AI's open code reconciles cloud findings only from successful scans ("Only successful scans produce truth") and carries services it did not scan forward as unknown (reconciliation). Secureframe shows each connection's sync status with the raw error, including "Partially Successful" (Secureframe), and Thoropass marks each monitor Healthy, Flagged or Connection Error (Thoropass). Probo's connector protocol covers

OAuth2, API keys, GitHub Apps and workload identity, with reconnect and probe capabilities (capabilities.go). Its access-review drivers are tested against 131 recorded API sessions, and a test fails if any recording contains an email outside its synthetic test domains (cassette test).

- **Gap.** CCP cannot tell an account that was removed from one it failed to read. So in live mode it never records a revocation, and if it treated every absence as removal, one failed page would mark live access as revoked. The vendors' replace-and-delete model would erase the history CCP exists to keep. Without a lifecycle and a test harness, every new connector brings these failures back.
- **Recommendation.** Make completeness part of the contract. Each pull records a sync run per data type that ends complete, partial or failed. Only a complete run may turn an absence into a revocation, stored as a new observation, never a delete. Add the connection lifecycle and a conformance harness before adding connectors.
- **How.**
  1. Contract: `pull()` returns events plus one sync-run record per data type (status, counts, pages, error class, cursor), landed in the raw zone and the event log like any other event.
  2. Land a snapshot only when its content changes, keyed by a hash of the payload without the pull time, and record "still present at time T" once per complete run instead. HR-1 and GH-1 judge freshness by a grant's latest observation (`ccp/evaluators/deterministic/hr1.py:4-9`, `ccp/evaluators/deterministic/gh1.py:50-51`), so they must read presence from the run record.
  3. Revoke only from complete runs: the mapper emits a `revoked` observation for each grant missing from a complete run of its data type. Partial and failed runs add and update but never revoke, and tests that depend on removals return `needs_review` while their data type's last run is incomplete.
  4. Harden the client: retry 429 and 5xx with backoff and jitter, honor `Retry-After` and GitHub's rate-limit headers, fail one data type rather than the whole pull, raise on GraphQL errors instead of returning nothing, and isolate connectors from each other. Test each failure with recorded responses.
  5. P1, lifecycle: a connections table per tenant; credentials held by a broker, never in settings; a GitHub App installation instead of a personal token; token refresh; health derived from sync runs (healthy, degraded, auth failed, stale); a schedule per connection; and intake for the membership webhooks the fixture already models (`fixture.py:30-35`), with complete snapshots as the backstop.
  6. P1, harness: record and scrub provider responses (normal, empty, page boundary, 401/403, 429, 5xx mid-listing, timeout), replay them against every connector, and assert that a failed listing never ends a run as complete. Require it for every new

connector. A model may draft a connector offline, but the harness and a reviewer decide whether it ships.

- **Tradeoffs.** Stricter runs mean more `needs_review` verdicts while a connector is partial, and customers will see them. A GitHub App adds an install step for each customer. Recorded responses drift as provider APIs change, so they need refreshing.
- **Priority.** P0 for steps 1–4 (about 3–4 engineer-weeks, estimate): a live pull today can keep failing a fixed control or hide an account. P1 for steps 5–6 (about 6–8 engineer-weeks, estimate).

D2. Citation checks confirm presence, not support (P0)

- **Today.**
  - A quote passes if it appears in the cited chunk, or if 85% of its words appear anywhere in the chunk, in any order and ignoring negation (`ccp/evaluators/citations.py:36-47`).
  - Only some fields must also appear in their quote: dates and providers for HR-3, timeline text for GH-4 (`ccp/evaluators/extraction/__init__.py:30-32`, `:38-40`; `citations.py:112`). Any true-or-false value passes that check (`citations.py:50-52`). So the fields that decide pass or fail, such as `completion_status`, `subject_name_in_document`, `policy_exists` and `timeline_stated`, accept any quote found in the chunk.
  - The repo's own fixture shows the risk. A training record reads "Status: In progress (3 of 5 modules completed)" (`fixtures/acme/hris/documents/training-E1044.md:12`). The probe recorded in A.7 found that a model answer of `completed`, citing that line, verifies, and the HR-3 rule then passes (`ccp/evaluators/rules/hr3.py:51-69`).
  - Tier 3 sets its retrieval factor to 1.0 unless its fallback search loop ran (`ccp/agent/graphs/tier3.py:189`), and it counts a list as covered if it names anything (`ccp/agent/core.py:451-453`). If the model leaves a critical system out of the inventory, HR-4 has nothing to call missing and can pass (`ccp/evaluators/rules/hr4.py:23-28`, `:46-56`) at confidence 1.0, above the 0.70 review gate (`tier3.py:210-217`; `config/evaluation.yaml:5`). Its verifier also accepts quotes from any chunk retrieved during the run, not only the chunks shown to that call (`core.py:428-429`).
  - The Copilot checks only its citation list. It drops failed citations but keeps the answer text as written (`ccp/agent/graphs/copilot.py:372-387`, `:429-436`).
  - `spec.md` A-17 defers stronger checks until support precision is measured (`spec.md:2237`). Nothing has measured it against a real model (D7).

- **Competitors.** No vendor documents citations checked in code. The closest is Sprinto: its August 2026 questionnaire pipeline separates fact extraction from answer writing and adds a grounding check with per-answer sources, but does not document how the check works (Sprinto). Vanta checks evidence with an LLM verifier that it measures on expert labels and against auditors' accept-or-flag decisions (Vanta). Drata's vendor-risk agent shows the source excerpt behind each criterion (TPRM Agent). Zania claims citations and confidence scores but publishes no verification step (Zania), and Probo's vendor-vetting tools store document URLs with no field linking a fact to its source (Probo). After the 2026 allegations against Delve, which Delve denies, evidence provenance is the category's credibility test.
- **Gap.** Citations checked in code could be CCP's clearest lead, because no competitor documents them. Today the check marks false passes as verified. That is worse than no check, because an auditor reads "verified" as proof.
- **Recommendation.** A field counts only when its quote supports its value. Anything the checker cannot confirm becomes `needs_review`, never a pass.
- **How.**
  1. Require an exact or contiguous match after whitespace is normalized. Remove the 85% word-overlap fallback and any splicing across sentences.
  2. Add a typed check for every field that decides the result: statuses from a fixed vocabulary in which "in progress" can never support "completed", dates parsed and compared exactly, names matched to the subject, and true-or-false fields backed by an explicit phrase. Negation words send the field to review.
  3. Check each quote only against the chunks shown in that call, and only against the evidence type the field requires.
  4. Tier 3: stop fixing the retrieval factor at 1.0, and add a recall check for lists. Parse the critical-systems table in code, or run a second extraction, and send any disagreement to review.
  5. Copilot: label sentences that no verified citation supports, or drop them.
  6. Test on 200–300 labelled field-and-quote pairs that include splices, swapped dates and negations, and let CI allow zero false accepts on them. Add HR-3 and HR-4 attack cases; all 16 today target GH-4 (D7).
- **Tradeoffs.** Automatic pass rates will fall and review queues will grow. Strict matching rejects some honest quotes broken by PDF line breaks or scanning noise, so normalization needs care. Each evidence type needs its own vocabulary, and labelling needs a named domain expert's time.
- **Priority.** P0: no AI-tier verdict should count until this ships. About 3–4 engineer-weeks plus 1–2 weeks of expert labelling (estimates); the labelled set also serves D7.

D3. Approvals can fail open (P0, then P1)

- **Today.**
  - `ccp approvals approve` takes `--decision` as free text, defaulting to `approve`, and never checks it (`ccp/approvals/cli.py:88-90`). The tier-3 gate treats any action other than `override` or `regather` as approval (`ccp/agent/graphs/tier3.py:231`, `:244-248`), so `reject` or a typo approves the draft verdict. `ccp agent resume` does check the value (`ccp/agent/cli.py:126-127`).
  - The approval is then recorded as `approved`, whatever the resumed graph did (`ccp/approvals/actions.py:51-54`). `ccp agent resume` writes a human-decided verdict with no approval event at all (`ccp/agent/cli.py:143-145`).
  - Resume checks only the tenant and that the thread is waiting (`ccp/agent/tier3.py:234-240`). The interrupt's approval item stores a payload hash (`ccp/approvals/store.py:79-88`), but nothing compares it with the checkpoint being resumed; only remediation checks a hash (`ccp/remediation/gates.py:77-83`).
  - `ccp remediate rollback` re-adds a removed collaborator with no gate check, no `--live` flag and no typed confirmation (`ccp/remediation/cli.py:111-117`; `ccp/remediation/runner.py:243-289`). It uses the permission saved before removal, or `write` when none was saved (`runner.py:269`).
  - A reviewer is any non-empty typed name (`ccp/approvals/store.py:101-102`).
  - What works: one append-only approvals table with a payload hash, dry run by default, and five gates on the one live action (`gates.py:53-84`).
- **Competitors.** Drata's vendor-risk agent asks before each write by default (TPRM Agent). Vanta's Auto Mode pre-approves eligible agent actions, but "destructive actions continue to require individual approval" (product updates), and its MCP remediation flow asks before running CLI changes against live resources (Remediate with MCP). Scrut requires an `Idempotency-Key` on every API write, so a retry cannot apply twice (Open API). Probo's open code rejects approval decisions that do not cover exactly the pending tool calls, locks the run while it records them, and lets only owners and admins approve (approval.go, service.go, policies.go), yet no production agent turns approvals on (agent.go). Comp AI's approval is the string `acknowledged`, after which another model call rewrites the fix steps (remediation), and its rollbacks skip its command denylist (executor), much as CCP's rollback skips its gates.
- **Gap.** In our reading CCP has the strongest approval contract on paper. In code it fails open in three places, and a typed name is not an identity, so an auditor cannot tell who approved what. A rollback writes to a customer's system like any other action.

- **Recommendation.** Treat approvals as data the server checks: a closed set of decisions, a payload hash checked when the approval is used, every write behind the same gates (rollback included), and a signed-in reviewer.
- **How.**
  1. Check the decision value in one shared function on every path. An unknown value fails, and `reject` resumes as `needs_review`, as `actions.reject` already does (`actions.py:68-78`).
  2. On resume, recompute the interrupt payload's hash from the checkpoint and refuse if it differs from the hash the approval item already stores. Lock the thread row so two resumes cannot both run, and write the approval event in the same transaction as the verdict, on every path.
  3. Route rollback through the gate check, with its own approval and typed confirmation. Refuse it when no pre-removal permission was saved, instead of granting `write`.
  4. Test each hole: a typo in the decision, a changed payload, two resumes at once, and a rollback without approval.
  5. P1, with D8's sign-in: reviewers are signed-in users with a reviewer role. For actions that change a customer's system, the reviewer must differ from the requester. The apply step is keyed by proposal id and payload hash, so a retry cannot apply twice.
- **Tradeoffs.** Operators get more steps, and an urgent rollback becomes slower. If speed matters, give rollback a separate emergency role that is logged and reviewed afterwards, rather than skipping the gates. Step 5 waits for user identity (D8).
- **Priority.** P0 for steps 1–4 (about 1.5–2 engineer-weeks, estimate): today a typo approves. P1 for step 5 (about 1–2 engineer-weeks once D8's sign-in exists, estimate).

D4. Background work is safe only as one process (P0, then P1)

- **Today.**
  - A verdict can be lost between two transactions. `commit_verdict` inserts the verdict, then announces it on the `verdicts` topic in a second transaction (`ccp/evaluators/runner.py:93-106`). If the process dies in between, a retry finds the evaluation key taken and skips the announcement (`runner.py:95-96`; `ccp/evaluators/verdict.py:102-107`), so the verdict never reaches compliance state or the Investigator. No code compares verdicts with state.
  - One bad record stalls an ordered topic. An exception in `handle()` rolls back the whole batch and leaves the cursor in place, so the same record fails on every run (`ccp/ingest/consumers.py:61-72`). Only parallel topics dead-letter

(`consumers.py:108-136`). The worker loop catches nothing, so the error also ends the process before later tenants run (`ccp/ingest/worker.py:86-103`). No code reads the dead-letter table or re-drives a record.

  - The Investigator is an ordered consumer on `verdicts` (`ccp/investigation/runner.py:208-212`). It raises the park signal when the model key is missing or the budget is spent (`ccp/investigation/runner.py:82`, `:237-238`), but only parallel consumers handle parking (`consumers.py:101-107`), so the Investigator stops at that record instead, like any other ordered-topic failure. It also calls the model while its batch transaction is open (`ccp/investigation/runner.py:239`, with `consumers.py:62-68`), against the rule in `ccp/ingest/topics.py:3-4`.
  - Several processes would skip rows. Offsets are numbered when a row is inserted but become visible when it commits (`ccp/storage/migrations/0001_schema.sql:37`). An ordered consumer reads past its cursor and jumps to the highest offset it saw (`ccp/ingest/event_log.py:58-66`; `consumers.py:69-71`), so a row that commits late with a lower offset is skipped for good. The worker runs one consumer per process (`ccp/cli.py:552-563`), so the change-driven path needs several processes, and the three normalize consumers all write to the `entities` topic (`ccp/normalize/consumer.py:78-85`, `:91-94`).
  - Several processes could also let a stale verdict replace a fresh one. Claims ignore the partition key, so two evaluations of one subject can run at once (`event_log.py:85-101`). The state writer makes whichever verdict arrives last the head of the chain, even an older one, which it labels `late_event` (`ccp/compliance_state/writer.py:44-51`, `:67-78`). The single-process `ccp run` avoids both problems (`ccp/compliance_state/run.py:49-78`).
  - One model budget parks deterministic tests. The daily budget is checked before the tier is known (`ccp/evaluators/runner.py:151-153`, `:165-166`). Each tenant's cap is $5 (`config/tenants.yaml:10`, `:18`), and tier 2, tier 3, the Investigator and the Copilot all charge the same ledger (`ccp/evaluators/tier2.py:287`, `ccp/agent/tier3.py:264`, `ccp/investigation/runner.py:117`, `ccp/copilot/runner.py:257`). Once it is spent, tier-1 tests park too. Only tests call `requeue_parked` (`ccp/evaluators/runner.py:231-236`), although the budget module says parked records resume when the day rolls over (`ccp/evaluators/budget_gate.py:1-3`).

- **Competitors.** The incumbents do not document their queues. A Drata job posting, removed in Sep 2026, listed Temporal (posting). Vanta lists Hatchet workflows among its subprocessors (subprocessors), and a Vanta AI-platform posting asks for durable execution with "checkpointing, retries, human-in-the-loop steps" (posting). Probo's

open code runs agents on Postgres much as CCP does, but each claimed run carries an owner token and a heartbeat lease that recovers stale runs, and a checkpoint is saved after every tool turn (handler.go, run.go). Drata's engineering blog argues that "Durability has to cover the decision, not just the execution" (Drata); that is strategy, not shipped architecture.

- **Gap.** The queue design is sound: claims, attempts, dead-letter, parking and budgets. The defects lose or stall work without telling anyone. A lost verdict leaves compliance state wrong, and a stalled topic looks the same as a quiet day.
- **Recommendation.** Fix the single-process defects now. Make the queue safe for several processes before the first customer needs a second worker. Choose a durable workflow engine only after that, and only if the hardened queue still falls short.
- **How.**
  1. Write the verdict and its announcement in one transaction. Add a reconciler that announces any verdict with no state row and reports how many it found.
  2. Give ordered topics a poison-record path: after a set number of failures, move the record to dead-letter with its error, advance the cursor and raise an alert. Catch errors per tenant in the worker loop, and ship commands to inspect and re-drive dead letters.
  3. Move the Investigator to the parallel `investigations` topic, which is declared but unused (`topics.py:25`), so its model calls run outside a transaction and parking works.
  4. Check the budget only before model calls, never for tier 1. Re-queue parked records when the day rolls over, and ship a command to do it by hand.
  5. P1, before a second process: number offsets per tenant and topic from a counter row locked in the producing transaction, as `tenant_sequences` already does for state (`ccp/storage/bitemporal.py:83-92`). Allow one evaluation in flight per subject, and file a late verdict into history without displacing a newer head.
  6. Test two producers whose commits interleave, a poison record, and a crash between a verdict and its announcement.
- **Tradeoffs.** A counter row makes producers on one topic take turns, which caps throughput; at CCP's scale that does not matter yet. One transaction per step means the verdict and its announcement must share a connection. A durable engine such as Temporal or Hatchet would add servers and upgrades to run.
- **Priority.** P0 for steps 1–4 and 6 (about 2–3 engineer-weeks, estimate): even one process can lose a verdict or stall. P1 for step 5 (about 2–3 engineer-weeks, estimate).

D5. Untrusted content and secrets (P0, then P1)

- **Today.**

  - Injection handling only detects. Evidence enters prompts inside a random fence, and a scan looks for 15 fixed English phrases (`ccp/security/prompt_guard.py:28-44`). A hit still shows the chunk (`prompt_guard.py:117-120`). In tier 2 it only caps confidence at 0.60 and leaves the status alone (`ccp/evaluators/confidence.py:76-78`; `ccp/evaluators/tier2.py:309-317`), and tier 2 has no review gate: the 0.70 threshold exists only in tier 3 (`ccp/agent/graphs/tier3.py:212-215`).
  - Tool results are not fenced. They go back to the model as raw JSON (`ccp/llm/client.py:126-140`, via `ccp/agent/core.py:367`). Only the Copilot scans them, and a hit only adds an event (`ccp/agent/graphs/copilot.py:350-357`).
  - Secrets are redacted only on the way into prompts and tool results (`prompt_guard.py:110-112`; `ccp/agent/tools.py:192-196`). Chunk text goes unredacted to the embedding API (`ccp/retrieval/index.py:44-45`, `:139`, `:151`) and the rerank API (`ccp/retrieval/hybrid.py:60-61`, `:231`). None of the six patterns covers an AWS secret access key (`ccp/security/secrets_patterns.py:11-20`), and raw payloads are stored as plain JSON (`ccp/ingest/raw_zone.py:62-71`).
  - GH-5 passes when no pattern matches (`ccp/evaluators/deterministic/gh5.py:51-57`), but the live client lists only top-level files and reads only those of 20,000 bytes or less (`ccp/connectors/github/client.py:206-207`, `:213`). Larger files become "(content not inlined)" (`ccp/normalize/github_mapper.py:282-284`), and files in subfolders are never listed, so a repository can pass without all its files being read.
  - Upload paths are not contained. A manifest entry's file name is joined to the tenant's folder without a check (`ccp/connectors/manual/uploads.py:88`), and the tenant comes from the folder (`uploads.py:3`). An entry such as `../globex/report.pdf` is read and stamped as the wrong tenant's evidence. Only the operator writes manifests today.
  - What holds: tools take the tenant from code, row-level security sits underneath, code decides every status, and every action needs approval (A.4, A.6).
- **Competitors.** Cross-tenant mistakes are the failures that have hurt vendors. Vanta's Inc-868 RCA says a code change removed a "domain ID filter", so data pushed through third-party integrations landed in other customers' tenants (RCA); the fix added database-layer protections for those integrations (trust updates). Comp AI shipped "P0 auth and tenant-scoping" fixes on 23 Sep 2026 (commits), and Probo's MCP tools take the organization id from the model, then check the user's rights (resolver). On

injection, Vanta lists Lakera and E2B sandboxes among its AI subprocessors but does not document how it uses them (subprocessors). None of the vendors we researched documents how it fences retrieved text or tool results.

- **Gap.** CCP's structure limits the damage: an injected document cannot approve an action or write a status directly. But detection is the only thing between an injected document and a tier-2 verdict, secrets leave CCP through two vendor calls, and one unchecked path can cross tenants, the kind of mistake that hurt Vanta.
- **Recommendation.** Contain first and detect second. Fence every untrusted input, tool results included. Send injection hits in the AI tiers to review. Keep secrets out of every outbound call. Resolve every file path inside its tenant's folder.
- **How.**
  1. Resolve each manifest path and refuse any that leaves the tenant's folder, including absolute paths, `..` and symbolic links. Add a test with an entry that points at another tenant.
  2. Redact before embedding and reranking with the same function the prompts use. Add patterns for AWS secret keys and a check for long random-looking strings. Encrypt the raw zone when it moves to object storage (D8).
  3. Make GH-5 return `needs_review`, naming the files, when any file was not read.
  4. Fence tool results with the call's random marker, as evidence already is, and scan them in every graph. In tier 2, make an injection hit force `needs_review`, as missing evidence already does (`confidence.py:68-71`).
  5. P1: switch off action-proposing tools for the rest of a run once it has read untrusted text; record each chunk's source, uploader, hash and trust level; and widen detection beyond 15 English phrases, as a signal only.
- **Tradeoffs.** Injection hits will send some honest documents to review, such as a security policy that quotes an attack phrase as an example. Redaction can lower retrieval quality for chunks that lose words. The rule in step 5 that switches off action tools will end some agent runs early.
- **Priority.** P0 for steps 1–4 (about 2–3 engineer-weeks, estimate): real customer files could cross tenants or reach a vendor unredacted. P1 for step 5 (about 2–3 engineer-weeks, estimate).

D6. Replay and change propagation (P0, then P1)

- **Today.**
  - The change-driven path skips identity resolution and indexing. The registered consumers are normalize, impact, evaluations, state, clock and investigator (`ccp/normalize/consumer.py:91-94`, `ccp/evaluators/impact.py:252`, `ccp/evaluators/runner.py:187`, `ccp/compliance_state/writer.py:122`,

`ccp/evaluators/clock.py:83`, `ccp/investigation/runner.py:248`). Only a full run and manual commands resolve identities and refresh the index (`ccp/compliance_state/run.py:65-67`; `ccp/cli.py:189`, `:238`, `:465`). A GitHub account first seen after the last full run stays unlinked, the impact step finds no terminated person in its group and queues nothing (`impact.py:141-146`), and HR-1's and GH-1's last pass stands until the next full run.

- New evidence is not indexed until a full run either. A tier-2 test triggered by that evidence can find nothing and record `needs_review` under that trigger's key (`ccp/evaluators/tier2.py:143-150`, `:188-205`). Re-running the same trigger cannot replace it, and neither can a full run while that evidence is still the tenant's newest observation, because the full run reuses that time as its trigger (`ccp/evaluators/runner.py:198-200`).
- The impact table is code. Test ids are hard-coded in `requests_for_entity` (`impact.py:149-209`), although the spec says "The mapping is a table, not code branches" (`spec.md:1146-1147`). A test left out is never triggered by a change.
- Verdicts store no rule version. The evaluation key covers tenant, test, subject, trigger time, prompt version and model (`ccp/evaluators/verdict.py:72-90`); deterministic verdicts pass neither of the last two (`ccp/evaluators/context.py:82-90`), and no verdict records a rule version or its parameters. So a corrected rule or a changed SLA may not re-decide unchanged evidence. A full run uses the tenant's newest observation as its trigger (`ccp/evaluators/runner.py:198-200`); until something new arrives, it gets the same key and the new verdict is dropped as a duplicate (`verdict.py:102-107`). The staleness clock re-decides only after the test's revalidation interval, 7 to 90 days (`ccp/evaluators/clock.py:47-61`). Either way, history records an ordinary evaluation or stale evidence, never a rule change (`ccp/compliance_state/writer.py:44-51`).
- Replay breaks once a cited document changes. Each index refresh keeps only current evidence and deletes other chunks (`ccp/retrieval/index.py:76`, `:188-192`). Replay looks chunks up in that live index (`ccp/replay/rebuild.py:168-173`), so earlier tier-2 and tier-3 verdicts stop replaying, although the stored request still holds the fenced text. Tier-1 replay re-runs nothing and reports no difference (`rebuild.py:250-251`), and the raw zone discards each file's stored hash on read instead of checking it (`ccp/ingest/raw_zone.py:76-77`).

- **Competitors.** Vanta keeps pass/fail timelines plus test snapshots with raw JSON and, for API-introspection tests, the requests made during the run (test history, snapshot API). Drata stores each test run as an event with raw JSON (test details). Thoropass binds evidence to one audit, and connected documents cannot be refreshed while their request is with the auditor (evidence reuse, connected documents).

Secureframe ships changes to built-in tests as versioned updates, added after a countdown of 14 days by default (test activity), and Vanta's custom tests cannot be edited after saving (custom tests). Comp AI keeps its long tail of checks as data and snapshots each check before every change (schema).

- **Gap.** CCP's history design is ahead of these vendors, but a record that cannot be replayed, or that cannot say which rule decided, proves less than Vanta's snapshot rows. The change-driven path is CCP's freshness advantage, yet it misses the cross-system case it was built for: a terminated employee who still has GitHub access.
- **Recommendation.** Run identity resolution and indexing on the change-driven path now. Then make every verdict reproducible from stored inputs: versioned chunks, a rule version and parameters on every verdict, and the impact table as data.
- **How.**
  1. Resolve identities and refresh the index for each changed entity before the impact step maps it, and re-trigger HR-1 and GH-1 when a new link forms; a `person_links` change already maps to both (`impact.py:157-158`).
  2. P1: keep superseded chunks. Key chunks by document version and content hash, mark old versions superseded instead of deleting them, have replay read by version and hash, and check each raw file's hash when it is read.
  3. P1: add a rule version and the parameters in force to every verdict and to its evaluation key, so a rule change re-decides unchanged evidence and shows in history as a rule change. Before a rule change ships, preview the verdicts it would flip and give customers notice, as Secureframe's countdown does for its test updates.
  4. P1: move the impact table into YAML beside `tests.yaml` , generate `requests_for_entity` from it, and fail CI when any test has no trigger.
  5. P1: make tier-1 replay re-run the rule over the stored observations and compare.
- **Tradeoffs.** Versioned chunks grow storage and embedding cost. A rule version in the key means each rule change re-evaluates every affected subject, which costs model calls on the AI tiers. Resolving identity on every change adds work to the busiest path.
- **Priority.** P0 for step 1 (about 1–2 engineer-weeks, estimate): without it, a change-driven run can keep a terminated employee's pass. P1 for steps 2–5 (about 4–5 engineer-weeks, estimate).

D7. AI quality is unmeasured (P0)

- **Today.**
  - CI has never run. The repository has no git remote (`git remote -v` prints nothing), so neither `.github/workflows/ci.yml` nor the keyed `.github/workflows/nightly-evals.yml` has executed.

  - All 27 recorded eval runs (`evals/runs/`, 12–15 Sep 2026) were keyless. No case used a token or cost anything, and no judge result was recorded. Eleven run headers list the configured models, such as `voyage-3-large`, but every case that records its retrieval models names the local stand-ins (`ccp/retrieval/embeddings.py:74`, `ccp/retrieval/rerank.py:94`). The run files are git-ignored (`.gitignore:4`), so they exist on one disk only.
  - The nightly gate blocks on any single pass-to-fail flip, any rise in attack success and any latency breach, and in strict mode on any first failure (`ccp/evals/regress.py:36-41`; `.github/workflows/nightly-evals.yml:56-70`). Each case runs once per suite run. In our reading the gate will fire on noise: if 50 model-dependent cases each pass 95% of the time, at least one flips on about 92% of nights (1 − 0.95^50).
  - The judge runs on the same model as tier 2, `claude-haiku-4-5` (`ccp/config.py:73`, `:75`; `ccp/evals/runner.py:257`). All 16 adversarial cases target GH-4 (`evals/adversarial/cases.yaml`), so HR-3 and HR-4 have no attack cases.
  - Model versions (checked on Anthropic's pages, 9 Nov 2026). Tier 2 and the judge use the alias `claude-haiku-4-5`, and Anthropic's pinning guarantee "covers model IDs, not the convenience aliases" (Model IDs and versioning). The pinned ID, `claude-haiku-4-5-20251001`, is Active with a tentative retirement date of "Not sooner than October 15, 2026". For `claude-sonnet-5`, used by tier 3 and the agents (`config.py:74`, `:107-116`), the date is "Not sooner than June 30, 2027". Anthropic gives at least 60 days' notice before a retirement (Model deprecations). Haiku 4.5's date has passed, so in our reading a notice can now come at any time.
  - CCP sends `temperature: 0` on every tier-2 and judge call (`ccp/llm/anthropic_client.py:62`, `:72-73`, `:199`), using SDK 1.5.0 (`uv.lock:29-30`). API version `2026-10-21` removes `temperature` and returns a 400 for it (same deprecations page), and CCP treats a 400 as terminal (`anthropic_client.py:10-12`; `ccp/llm/errors.py:17-18`, `:28-31`). Moving to that version without removing the parameter would dead-letter every tier-2 call and fail every judged eval case (`ccp/evals/runner.py:269-270`).
- **Competitors.** Vanta publishes eval results: control-to-evidence mapping precision rose from below 35% to 78%, and judge agreement with experts from about 65–70% to about 85–90% (Vanta, Jun 2026). It also scores its evidence verifier against auditors' accept-or-flag decisions (Vanta). Sprinto reports a faithfulness score of 0.995 on an internal 30-question benchmark (Sprinto). Drata uses Braintrust only for internal, offline evaluation (subprocessors). In open source, CISO Assistant ships a

harness that measures how accurately its assistant picks tools as the number of tools grows (harness), while Comp AI and Probo ship no AI-quality evals. Entrants such as Zania claim citations and confidence scores but publish no evaluation method (B.7).

- **Gap.** Evals are where CCP is behind until it runs them, because nothing yet shows its AI tiers work on a real model. Published numbers are rare in the category, so a measured false-pass rate would set CCP apart. A gate that fires on noise will be switched off, and then nothing guards a model change.
- **Recommendation.** Run the suites for real before any verdict depends on a model: a private remote so CI runs, a keyed baseline with repeated runs, gates on confidence bounds rather than single flips, a judge from another model family, and pinned model IDs with a migration plan.
- **How.**
  1. Push to a private remote with branch protection, so CI runs on every change. Keep eval reports as CI artifacts or database rows, not ignored local files.
  2. Build the first keyed baseline: golden and adversarial suites on real models, 5 runs per case, recording pass^k (the chance that all k runs of a case pass) and verdict flips on unchanged evidence.
  3. Replace the single-flip rule. Compare versions on the same cases, block when the lower confidence bound drops below an agreed margin, and count errored runs as failures.
  4. Move the judge to a different model family and calibrate it on 100–200 human labels per failure mode. Require Vanta's published 85% agreement with experts before the judge gates anything.
  5. Have a named domain expert label sets for GH-4, HR-3 and HR-4, reporting false passes and false fails separately, and add HR-3 and HR-4 attack cases.
  6. Pin `claude-haiku-4-5-20251001`, store the model ID, API version and SDK version on each verdict, and remove `temperature` before any API-version change. Plan tier 2's move now, for example to `claude-haiku-5` (Active, not sooner than October 7, 2027, on the same deprecations page), behind a full paired comparison.
- **Tradeoffs.** Real runs cost money: the nightly job sets a $0.50 budget per run and a $5 cap on the whole job (`.github/workflows/nightly-evals.yml:39-41`), and five runs per case multiply the cost. A judge from another family adds a vendor and its data terms. Expert labelling is the slowest part and needs a named owner.
- **Priority.** P0: no verdict should rely on a model until this runs. About 3–4 engineer-weeks (estimate), plus expert labelling shared with D2.

D8. No product surface or operating model (P1, P2 for MCP)

- **Today.**
  - CCP is teaching-scale by its own spec (`spec.md:1`). It is a command line: its only interface dependency is `typer` (`pyproject.toml:12`). There is no API, UI, user sign-in or role-based access, and reviewers are typed names (`ccp/approvals/store.py:101-102`).
  - Tenants come from a YAML file (`config/tenants.yaml`; `ccp/config.py:50`, `:137`), and there is no onboarding or connection setup (D1).
  - The worker is a polling loop that runs one consumer per process and sleeps 2 s when idle (`ccp/ingest/worker.py:67-103`). No code raises an alert: dead letters and parked records show only in their tables, in log lines (`ccp/ingest/consumers.py:116-136`) and in the parked count that `ccp evaluate run` prints (`ccp/cli.py:202-207`).
  - The raw zone is a folder on local disk (`ccp/config.py:49`, `:133-135`; `ccp/ingest/raw_zone.py:52-72`). Postgres runs from `docker-compose.yml`, and the repo holds no deployment or backup setup. Nothing purges customer data: the app role cannot delete history by design (`ccp/storage/migrations/0002_rls.sql:9-13`), and there is no purge path for contract end or an erasure request, so the event log and raw zone grow without limit.
  - Every component uses one database role (`ccp/storage/db.py:34-36`), so agent tools are read-only by code review, not by a read-only role.
  - `spec.md:3` still says "No implementation code exists yet."
- **Competitors.** Vanta builds every product action as one authorized service method and exposes it through REST, its CLI, its agent and MCP (Vanta, Aug 2026). MCP servers range from Scrut's 19 read tools and 3 writes (Scrut) to Secureframe's 100+ tools with create, update and delete and no read-only mode (Secureframe). Drata limits MCP access to the intersection of granted scopes and the user's role (Drata). Probo lets only owners and admins approve agent actions, while viewers and auditors are read-only (policies.go). Drata shows auditors sampled evidence inside the audit window by default, or grants them read-only access to the whole application (audit sampling, auditors).
- **Gap.** The incumbents lead on breadth and product surface. CCP cannot onboard a customer, sign in a reviewer, show an auditor its record, delete a departing customer's data, or survive the loss of one disk.
- **Recommendation.** Build the smallest surface a first customer needs, from one typed list of actions that the API, the agents and later MCP all share: an API with sign-in, roles, tenant and connection onboarding, a review and approval screen, and a cloud

deployment with the raw zone in object storage.

- **How.**
  1. Make tenants and connections database rows, with an onboarding flow. Keep the YAML file for fixtures only.
  2. Grow the agents' read-only tool module (A.1) into that typed list, and generate the API from it, with sign-in through the customer's identity provider and roles for admin, reviewer and read-only auditor. Approvals bind to the signed-in user (D3 step 5).
  3. Build a small UI: connection health (D1), the review queue, approvals with the cited quotes highlighted, and verdict history with `state_as_of` for auditors.
  4. Deploy: containers for the API and workers under a supervisor, managed Postgres with backups, the raw zone in encrypted object storage with a retention lock, secrets in a secrets manager, and alerts on queue depth, dead letters and parked records.
  5. Add a retention and erasure path, run by a maintenance role that records each purge as an event, and a read-only database role for agent tools.
  6. P2: a read-only MCP server generated from the same list, where writes arrive only as approval requests.
  7. Correct `spec.md:3` now.
- **Tradeoffs.** This is the largest item, and it competes with D1–D7 for the same engineers. A UI shipped before D2 and D3 would show unverified results next to an approval button that fails open. Settle retention and erasure terms before turning on any lock that cannot be shortened.
- **Priority.** P1 (about 12–16 engineer-weeks for a minimal surface, estimate). P2 for MCP. The `spec.md` fix takes minutes.

E. Where agents add value

Agents fit where the input is unstructured text and a person makes the final call: questionnaires, connector drafting, framework mapping, uploaded evidence and remediation. They do not fit pass or fail, compliance status, sync health or identity matching. There, code is cheaper, gives the same answer every time and cannot be prompt-injected. Every agent below proposes, code verifies, and a person or a deterministic runner acts. CCP already runs four agents this way (A.4), so these five opportunities extend a working design. None should ship before the Section D fixes it needs. Code paths are relative to `ccp/` at `3386899`; `tests/`, `config/` and `.github/` sit at the repo root.

Shared rails, from CCP's own design (A.3, A.4):

- **Tools only read, and code binds the tenant.** Each tool takes its tenant from a closure and rejects extra arguments (`agent/tools.py:1-8`, `:35-36`). A call to a tool not on the graph's list gets an error (`agent/core.py:339-341`). Next comes one typed registry that marks every tool read, propose or act, with agent tools on a read-only database role. Today they share the application role, which can insert (`storage/db.py:34-36`; `storage/migrations/0005_state.sql:66`).
- **Models extract, code decides.** The model fills a typed schema in which each field cites a chunk and a quote. Code checks the quotes (`evaluators/citations.py:87-129`), and a pure rule makes the decision (`evaluators/rules/__init__.py:1-3`). Confidence is computed, never reported by the model, and missing evidence gives `needs_review` (`evaluators/confidence.py:1-15`). The citation check must test support, not just presence (D2).
- **Untrusted text stays data.** Evidence reaches a prompt only inside a per-call fence (`security/prompt_guard.py:105`). Tool results need the same treatment (D5).
- **Agents propose, a person approves the exact payload, and code acts.** The approval stores a sha256 of the payload (`approvals/store.py:22-24`), and the live gate refuses an edited proposal (`remediation/gates.py:77-83`). First the approval path must fail closed and use signed-in reviewers (D3, D8).
- **Every run is budgeted, recorded and replayable, and evals gate changes.** Runs have token, dollar, time and tool-call caps (`llm/budget.py:22-27`), every step is logged (`observability/steps.py:1-8`), and model-backed records can be replayed (`replay/rebuild.py:1-10`). But replay breaks once a cited document changes (D6). Golden, judge and adversarial suites exist (`config.py:58-62`), but none has run on a real model (D7). A workflow earns lighter review only on measured accuracy, consistency and reviewer agreement, and only for reversible actions (companion document, Section 4.4).
- **MCP starts read-only.** MCP is the protocol AI assistants use to call outside tools, and CCP has no MCP server yet. It should be generated from the same registry (M3), take the tenant from the signed-in session, and accept writes only as approval requests. D8 rates it P2. Vanta, Drata and Secureframe instead let agents act with the user's permissions (B.1–B.3).

Keep these deterministic: connecting accounts and collecting evidence, sync completeness and connection health, pass or fail and compliance status, identity matching, exception expiry, applying approved changes, and audit packages. Identity matching already uses logged scoring and a review queue, with no model (`identity/scoring.py:1-13`). Each item is a rule over structured data that an auditor must be able to re-run. The companion target-architecture document places all 14 workflows the same way (its Section 4.3).

#	Opportunity	Why an agent	Needs first	Priority
E1	Questionnaire answers with verified citations	Questions come in endless phrasings, and every answer must be written and cite evidence	D2, D5, D7; signed-in approval (D3, D8)	P2 (L1)
E2	Connector and check drafting behind a conformance harness	Provider docs are unstructured; the code follows a pattern that tests can check	D1 lifecycle, completeness record and harness; D6 for checks as data	P2 (L7)
E3	Framework mapping proposals	Requirement and control text is prose; a wrong mapping changes every status built on it	D6 versioned catalogue; signed-in reviewers (D8)	P2 (L5)
E4	Evidence review of uploads	Uploads are documents and scans; reading needs a model, judging does not	D2 and D7, already P0 because three tests rest on this today; object storage (D8)	P2 for new evidence types (L2)
E5	Remediation by pull request	The cause is not known in advance; the fix is a diff a person can read	D1, D3, D4, D5; signed-in reviewers (D8)	P2 (L3)

E1. Questionnaire answers with verified citations

- **Why an agent.** Security questionnaires arrive as spreadsheets and portals, in endless phrasings, and every answer must be written and backed by evidence. CCP has no questionnaire feature yet. Its closest parts are tier-2 extraction and the Copilot, whose answer citations are checked for presence only (`agent/graphs/copilot.py:364-383`). Build it as a fixed pipeline, not an open tool loop: retrieve, extract cited facts, set the answer by rule where a question maps to a control, then write. Sprinto runs similar passes with a grounding check, and reports 0.995 faithfulness on a 30-question internal benchmark (Sprinto, Aug 2026).
- **Tools and data.** A library of approved answers, append-only and versioned, each tagged with the controls it relies on. Policies and other evidence come through the existing retrieval index under row-level security

(`storage/migrations/0008_chunks.sql:55-56`), and current compliance state serves as a read-only cross-check. The questionnaire itself is untrusted input and is fenced like evidence (D5).

- **Validation.** Code checks every citation with the hardened checker (D2): a contiguous quote from a chunk shown to that call, with every deciding value inside it. Code blocks any answer whose tagged control is `fail` or `needs_review` in current state. An answer that cannot be verified becomes "needs input", never a guess. A signed-in person approves before anything is sent (D3, D8). Do not auto-approve "high confidence" answers, an option Drata's SafeBase offers (AIQA guide), until accuracy is measured (D7).
- **Metrics.** Share accepted without edits; citation survival rate; answers that contradict compliance state (target 0); reviewer minutes per questionnaire; cost per answer.

E2. Connector and check drafting behind a conformance harness

- **Why an agent.** Breadth is CCP's widest gap: three connectors, one of them live, against the 400+ integrations Vanta claims (B.1). A connector is pattern-following code written from unstructured API docs, and a check is a small rule over typed entities. An agent can draft both offline. Tests and an engineer decide what ships, and verdicts at run time stay deterministic. Drata used an offline pipeline to write 1,000+ deterministic tests, each validated against its own test API with up to 3 attempts and reviewed by auditors (Drata, Dec 2025).
- **Tools and data.** Provider API docs and the upgraded connector contract, with a connection lifecycle, cursors and a completeness record (D1). CCP's pure mappers and rule modules serve as examples, with recorded provider responses and sandbox accounts that CCP owns.
- **Validation.** A conformance harness, a test suite every connector must pass, replays recorded responses: empty listings, the last page, 401 and 403, 429, a 5xx in mid-listing and a timeout. It asserts that a failed listing never reads as empty, that a partial pull is marked incomplete, and that a non-compliant fixture fails. A live run against a sandbox account must succeed, and the requested scopes must cover every read and nothing more. An engineer reviews the code.
- **Metrics.** Engineer-hours per connector; harness pass rate of the first draft; defects in the first 30 days; share of drafted checks accepted.

E3. Framework mapping proposals

- **Why an agent.** CCP maps 9 controls to a 6-criterion SOC 2 slice and an 8-control ISO 27001 slice, by hand, in YAML (`knowledge/controls.yaml:4-50`, `knowledge/frameworks/soc2.yaml:6-51`, `knowledge/frameworks/iso27001.yaml:6-68`). A catalogue for real customers needs far more, and every new framework means

matching requirement text to controls. That is semantic search plus judgement. A requirement takes the worst state of its tests (`knowledge/rollup.py:1-9`), so a wrong mapping changes every status built on it, and a person must accept each change. Drata's Framework Mapping Agent works this way: it proposes with a confidence indicator, leaves weak matches unmatched rather than force a guess, and approves nothing on its own (Drata).

- **Tools and data.** Requirement and control text, existing mappings as labelled examples, embeddings over the controls, and a named compliance reviewer with budgeted hours. Start as an offline batch for the frameworks customers ask for.
- **Validation.** Before launch, hold out one mapped framework at a time and predict its mappings. Each proposal cites the requirement and control text, and nothing applies until a signed-in reviewer accepts it (D8). Each accepted change creates a new catalogue version, so an as-of query shows which mapping a status used (D6).
- **Metrics.** Precision and recall at 3 on held-out frameworks; reviewer acceptance rate; days to add a framework.

E4. Evidence review of uploads

- **Why an agent.** Much manual evidence is documents: policies, training records, access reviews, penetration-test reports and vendors' SOC 2 reports. Reading them needs a model; judging them should not. CCP already works this way for three tests. Tier 2 reads a disclosure policy (GH-4) and a training record (HR-3), and tier 3 reads an access review against a critical-systems list (HR-4). Because these verdicts already rest on model output, their fixes are P0 (D2, D7). The opportunity is to extend the pattern to more evidence types, and to scans and screenshots. Today an upload must be Markdown, text or a PDF with a text layer. A PDF without one is flagged `no_text_layer`, and other files are skipped (`connectors/manual/uploads.py:108-132`, `connectors/manual/pdf_text.py:8-13`).
- **Tools and data.** For each evidence type, a typed extraction schema, a pure decision rule and a freshness window, as tier 2 has today (`evaluators/extraction/__init__.py:25-41`; `knowledge/tests.yaml:50`, `:96`). Uploads are hashed, dated and kept in object storage (M5). Count the evidence types customers upload most before choosing which to build.
- **Validation.** Follow Vanta's verifier rubric: admissibility (dated, attributable, and showing the control was carried out) and adequacy (the right document for the control). Vanta measures its verifier on labelled cases and by agreement with auditors' decisions (Vanta, Aug 2026). A named compliance reviewer first labels a fixed sample, for example 50 per type. Run in shadow mode against those labels, recording each verdict without acting on it. Launch only below an agreed false-accept rate, for example 1%. For vendor reports, the risk owner makes the decision.

- **Metrics.** False-accept rate (target near 0); agreement with reviewers and auditors; stale evidence caught; reviewer minutes saved.

E5. Remediation by pull request

- **Why an agent.** When a test fails, the cause is not known in advance, so an agent must look through grants, history and code to find it. CCP's Investigator already does this, read-only, for every `fail` or `needs_review` verdict, and proposes actions from a closed list (`investigation/runner.py:1-5`, `knowledge/actions.py:4-11`). One action can run live, removing a GitHub collaborator, behind five gates (`remediation/gates.py:1-9`). The next step is a fix a person can read: a pull request in the customer's infrastructure code, opened by CCP's GitHub App with exactly the approved change. Drata also delivers fixes as pull requests against the customer's Terraform, and Vanta's MCP flow has the customer's own coding agent open a draft pull request (Drata, Vanta). The companion document walks through this flow for a public AWS bucket (its Section 6.2).
- **Tools and data.** The failing verdict and its evidence, the subject's history, canonical entities, and the customer's infrastructure repository, read through the tool registry (M3).
- **Validation.** The proposal cites the findings it fixes, and code checks those citations. A signed-in reviewer approves the exact diff, and the executor opens the pull request only if the payload hash still matches (D3). On this path CCP holds no write access to the customer's cloud. Success is judged by the next complete pull and a deterministic re-run, not by the agent. Today the loop closes only through a test-only fixture overlay (`tests/e2e/test_remediation_loop.py:1-4`), because a live pull cannot see a removal (D1).
- **Metrics.** Median time from fail to pass; share of pull requests merged unchanged; reopen rate within 30 days.

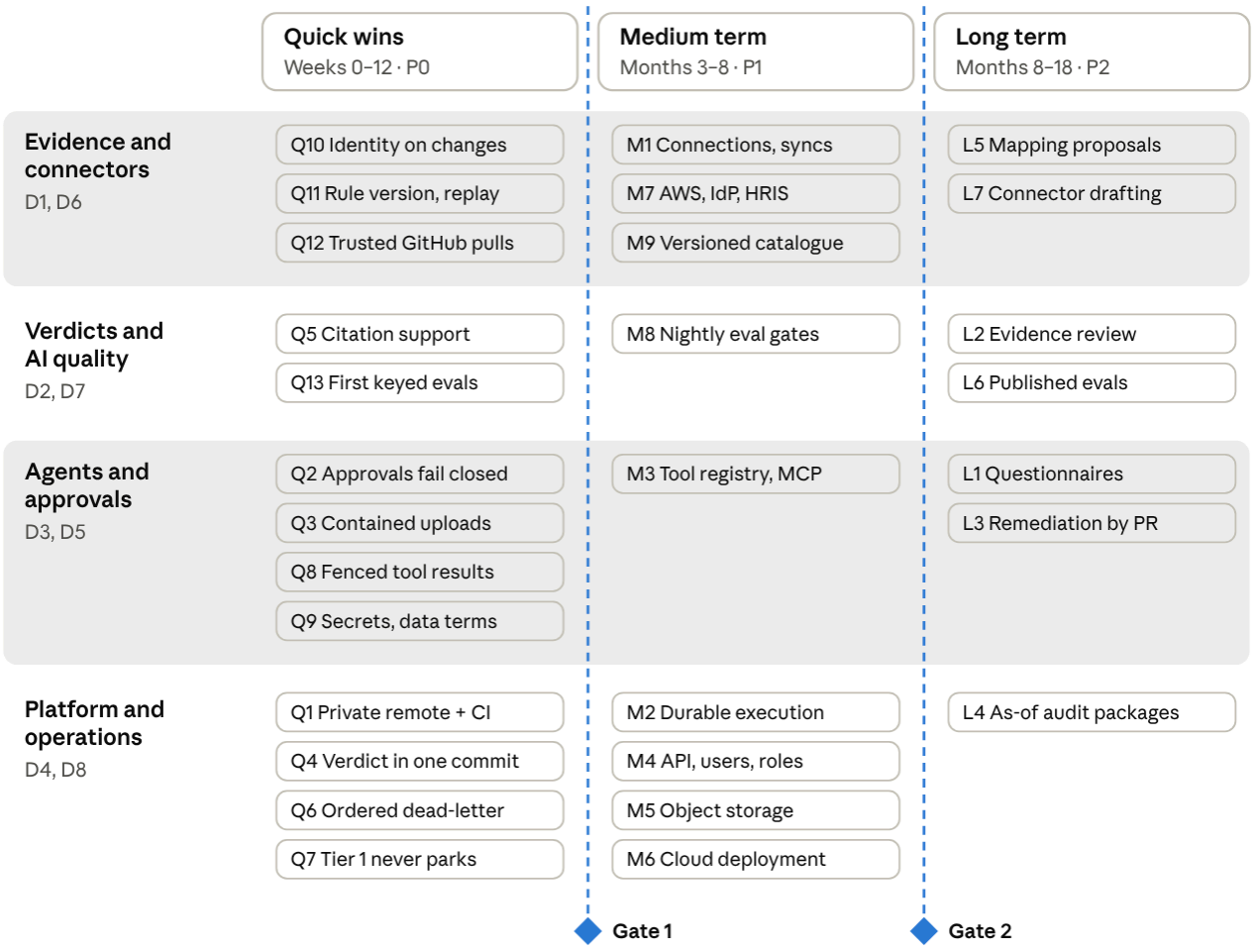
F. Prioritized engineering roadmap

Fix correctness before breadth. The plan assumes three engineers spending about two-thirds of their time on it, roughly 2 engineer-weeks per calendar week; the rest goes to support and other work. All efforts below are our estimates in engineer-weeks. At that rate the quick wins (about 17–24 engineer-weeks) take about nine to twelve weeks. They fix the defects Section D verified and make AI quality measurable. CCP needs both before it handles real customer data or relies on a real model's output (P0), and they add no new scope. Q11 is the exception: D6 rates it P1, but it is small and M9 builds on it. The medium term (about 38–52 engineer-weeks, months 3–8) builds what a first production customer needs (P1). The long term (months 8–18) builds the differentiators (P2). Its items total

about 29–40 engineer-weeks; the rest of that time goes to connectors drafted through L7, catalogue content, a read-only MCP server (D8) and customer requests. The companion target-architecture document designs the medium and long term in more detail.

Fix and measure first, then build the foundations, then differentiate

Workstreams by phase; phases are not to scale. Ids match the tables below.



Gate 1, end of quick wins: P0 defects fixed, CI green, false-pass rate measured

Gate 2, end of month 8: complete syncs, signed-in approvals, nightly eval gates

Efforts are estimates: quick wins about 17–24 engineer-weeks, medium term about 38–52, long term about 29–40.

Roadmap · 4 workstreams across 3 phases, 2 gates

Gate 1, before CCP takes real customer data or relies on a real model's output. Q2 to Q10 and Q12 are merged, each with a test that reproduces its defect. CI is green on the private remote (Q1), and data terms are in place (Q9). A keyed eval run has measured the false-pass rate on labelled cases, and it is below an agreed limit (Q13).

Gate 2, before long-term items and before a first production customer. Every live connector passes the harness and records completeness (M1, M7). Durable execution has passed its crash and deploy tests (M2). Every verdict records its rule version and still

replays after its documents change (Q11). Only signed-in reviewers with the right role can approve (M4). The raw zone is in object storage, and CCP runs in the cloud with backups and alerts (M5, M6). Nightly evals on real models gate every release (M8).

Suggested order: Q1 on day 1, then Q2, Q3 and Q4 in weeks 1–2. Q5 to Q8 follow in weeks 2–6, with a baseline keyed eval run (Q13) in week 2 to measure Q5′s effect. Q9 to Q11 run in weeks 5–8, and Q12 and the rest of Q13 in weeks 6–12. In the medium term, M2 and M3 start first, because M1, M4 and M6 build on them; M7 follows M1.

Quick wins (weeks 0–12)

#	Item	Complexity	Depends on	Customer value
Q1	Push the repo to a private remote, tag `3386899`, and turn on the CI that already exists (`.github/workflows/ci.yml:41-49`). Fix `spec.md` line 3, which says no code exists	Low · 0.5 wk	Nothing	Every later fix is tested on every change (D7, D8)
Q2	Approvals fail closed. Accept only known decisions (`approvals/cli.py:88-90`, `agent/graphs/tier3.py:231-248`). Record the decision before resuming (`approvals/actions.py:51-53`). Check the payload hash on every resume. Run rollback through the five gates (`remediation/runner.py:243-289`)	Medium · 1.5–2 wk	Q1	A typo can no longer approve, and only the approved payload runs (D3)
Q3	Refuse any upload whose manifest path resolves outside the tenant's folder (`connectors/manual/uploads.py:88`). Test `../`, absolute paths and links	Low · 0.5 wk	Q1	An upload can never be read into another tenant (D5)
Q4	Commit each verdict and its announcement in one transaction (`evaluators/runner.py:93-106`). Add a reconciler that announces any verdict missing from compliance state. Check for a stored verdict before calling a model	Low · 0.5–1 wk	Q1	No verdict is lost before compliance state and the Investigator see it (D4)

#	Item	Complexity	Depends on	Customer value
Q5	Citation support checks. Require a contiguous quote, with no 0.85 word-overlap match (`evaluators/citations.py:36-47`). Check the value of every deciding field, status included (`evaluators/extraction/__init__.py:32-57`). Accept quotes only from the chunks shown to that call (`agent/core.py:428-429`). Measure tier-3 retrieval quality instead of fixing it at 1.0 (`agent/graphs/tier3.py:189`), and check that extracted lists are complete (`agent/core.py:451-453`). Use the same checker in the Copilot. Gate on labelled quote cases at zero false accepts	Medium · 3–4 wk	Q1	A field counts only when its quote supports it, so the "In progress" training record no longer passes (D2)
Q6	A record that keeps failing on an ordered topic goes to the dead-letter table, raises an alert and can be replayed (`ingest/consumers.py:61-72`). Retries back off (`ingest/event_log.py:110-115`). The Investigator moves to the parallel `investigations` topic, where parking works (`investigation/runner.py:208-212` , `ingest/topics.py:25`). Tier 3 resumes its thread on retry instead of starting over (`agent/tier3.py:157-158`)	Medium · 1–1.5 wk	Q4	One bad record no longer stops a tenant's topic, the Investigator's included (D4)
Q7	Check the model budget after routing, so tier-1 tests never park (`evaluators/runner.py:151-153`). Re-queue parked records when the day rolls over, as the budget module says they will (`evaluators/budget_gate.py:1-3`), and add an unpark command over `requeue_parked` (`evaluators/runner.py:231-236`). Refuse models missing from the price table instead of pricing them at $0 (`observability/cost.py:43-50`)	Low · 0.5 wk	Q1	Deterministic results keep flowing, and spending caps work (D4, D7)
Q8	Fence and scan tool results in all four agent loops, as evidence already is (`llm/client.py:126-140`). A marker hit sends the result to review, and in tier 2 it forces `needs_review` instead of only capping confidence (`evaluators/confidence.py:76-78`)	Medium · 1–1.5 wk	Q1	Text planted in a repo file or an upload cannot steer an agent unseen (D5)

#	Item	Complexity	Depends on	Customer value
Q9	Redact secrets before embedding and rerank calls (`retrieval/index.py:139-151` , `retrieval/rerank.py:59-65`). Detect more formats, such as AWS secret keys (`security/secrets_patterns.py:11-20`). GH-5 returns `needs_review` when it could not read every file (`evaluators/deterministic/gh5.py:51-57`). Opt out of Voyage model training (MongoDB) and settle retention terms with each model provider	Low · 1–1.5 wk	Q1	Customer secrets stay out of third-party APIs and their training data (D5)
Q10	Resolve identities and refresh the index on the change-driven path, not only in full runs (`compliance_state/run.py:65-67`). Fail CI when a test has no trigger in the impact table (`evaluators/impact.py:149-209`)	Medium · 1–1.5 wk	Q4	GH-1 and HR-1 cannot pass for a terminated employee whose account appeared after the last full run (D6)
Q11	Store the rule version on every verdict (`storage/migrations/0005_state.sql:6-33`). Replay from the fenced text stored with each request, not the live index (`replay/rebuild.py:168-173`). Make tier-1 replay re-run the rule over the stored observations (`replay/rebuild.py:250-251`)	Medium · 1.5–2 wk	Q1	Every verdict can be rebuilt and explained after a document or rule changes (D6)
Q12	Trustworthy live GitHub pulls. Retry with backoff, respect rate limits, and raise a failed email lookup as an error (`connectors/github/client.py:55-60` , `:80-81`). Derive snapshot ids from content (`:117-130`). Record whether each pull of each data type was complete. A member missing from a complete pull becomes a revocation, written as a new observation (`normalize/github_mapper.py:303` , `:321`)	Medium · 3–4 wk	Q6	Live results match GitHub, and the remediation loop closes without the test-only overlay (D1)

#	Item	Complexity	Depends on	Customer value
Q13	First keyed eval run. Pin dated model ids. Remove `temperature` (`llm/anthropic_client.py:62-73`), which API version 2026-10-21 rejects (Anthropic). Run the judge on a different model family from tier 2 (`config.py:73-75`). Run the suites on real models, five runs per case, before and after Q5. Include `claude-haiku-5-5` , so a retirement notice for Haiku 4.5 never forces an unmeasured switch. Gate on confidence bounds, not any single flip (`evals/regress.py:1-5`). Add HR-3 and HR-4 attack cases; all 16 today target GH-4 (repo root, `evals/adversarial/cases.yaml:9-177`). Grow the judge's 20-case calibration set (repo root, `evals/labeled/README.md:3-5`)	Medium · 2–3 wk, plus a reviewer's labelling time	Q1; Q5 for the second run	The first measured false-pass rate and verdict stability for tiers 2 and 3 (D7)

Medium term (months 3–8)

#	Item	Complexity	Depends on	Customer value
M1	Connection lifecycle and sync completeness. Each connection gets a record, stored and refreshed credentials, a health state and a schedule. A cursor lets a pull resume where the last one stopped. Every pull records whether it was complete, and only a complete pull can turn an absence into a removal. Every connector must pass a conformance harness	High · 6–8 wk	Q12; M2 for schedules	Live evidence is complete, current and visible per connection (D1)

#	Item	Complexity	Depends on	Customer value
M2	Durable execution. First make consumers safe to run side by side (`ingest/consumers.py:61-72`, `compliance_state/writer.py:67-77`). Then test crashes and deploys: kill a worker mid-step, overlap two deploys, fail over the database. If the hardened queue fails, move to DBOS Transact inside CCP's workers, with Hatchet second and Procrastinate as the fallback (companion document, Section 3.5). A run ends when it stores a proposal, and an approval starts a separate apply step that is safe to repeat	High · 4–6 wk	Q4, Q6	Work survives crashes and deploys, and runs on more than one process (D4)
M3	Typed tool registry and policy check. Each tool declares typed inputs and outputs, a class (read, propose or act), scopes, a rate limit and a timeout. Missing data is an error, never an empty list. Agent tools get a read-only database role (`storage/db.py:34-36`), and one policy check runs before every call. Once a run has read untrusted text, the check switches off its propose and act tools. Each chunk records its source, uploader, hash and trust level. A read-only MCP server comes later from the same registry (P2, D8)	Medium · 4–6 wk	Q8	One contract for agents, the API and later MCP, and a tool bug cannot write (D5, D8)

#	Item	Complexity	Depends on	Customer value
M4	API, signed-in users and roles. Customers sign in, with roles per tenant: admin, reviewer and read-only auditor. Authenticated reviewers replace typed names on approvals and identity reviews (`approvals/cli.py:86`, `cli.py:479`). Tenants move from YAML into the database (`config/tenants.yaml:1-3`). The API is generated from the registry, with review screens for approvals, identity matches and `needs_review` verdicts	High · 6–8 wk	Q2, M3	Every decision has an accountable person, and customers no longer need the command line (D3, D8)
M5	Raw zone in encrypted object storage, under tenant prefixes, with versioning and hashes checked on read. A retention lock in governance mode lets only an emergency role delete early, because raw data can hold secrets uploaded by mistake. A purge path for contract end and erasure requests records each deletion as an event (D8)	Medium · 3–4 wk	Q3; retention terms agreed	Evidence survives the loss of a server and cannot be quietly rewritten (D8)
M6	Cloud deployment: containers, managed Postgres with backups and point-in-time restore, a managed secrets store, separate worker processes, and alerts on dead-lettered and parked work	Medium · 3–4 wk	M2, M5	CCP runs without a developer's machine and recovers from failures (D8)
M7	First production connectors on the new contract. AWS through a read role the customer deploys, with an external ID (companion document, Section 6.2). An identity provider keyed by its immutable user ids. HRIS data through an aggregator, as Vanta and Drata use Merge (B.1, B.2)	High · 6–8 wk	M1	A first customer's cloud, people and access are covered (D1)

#	Item	Complexity	Depends on	Customer value
M8	Eval gates on real models. Nightly runs gate on confidence bounds, and labelled sets grow toward 100–200 cases per failure mode. A judge gates nothing until it agrees with experts at least 85% of the time, the level Vanta reports reaching (Vanta, Jun 2026). Then tier 2 moves off Haiku 4.5 through these gates; the judge already left it in Q13. Haiku 4.5's earliest retirement date, 15 Oct 2026, has passed, and Anthropic gives at least 60 days' notice (Anthropic)	Medium · 3–4 wk, plus labelling	Q13	Model and prompt changes ship only when measured quality holds (D7)
M9	Versioned catalogue. Frameworks, controls, tests and the impact table become versioned data, not code (`evaluators/impact.py:149-209`). Rule changes ship with notice, as Secureframe's versioned test updates do (B.3), plus a preview of the verdicts they would change	Medium · 3–4 wk	Q10, Q11	Tests can be added without code changes, and customers see a rule change before it moves their status (D6)

Long term (months 8–18)

#	Item	Complexity	Depends on	Customer value
L1	Questionnaire answers with verified citations: a fixed pipeline, a library of approved answers, and a person who approves what is sent	High · 6–8 wk	Gate 2; M3	Answers a buyer can trust, each traced to its evidence (E1)

#	Item	Complexity	Depends on	Customer value
L2	Evidence review for more upload types, including scans, screenshots and vendors' SOC 2 reports, in shadow mode against reviewer labels first	High · 5–7 wk	M5, M8, a labelled sample	Faster reviews; stale or wrong evidence caught before an auditor sees it (E4)
L3	Remediation by pull request through CCP's GitHub App, opened only from an approved payload and confirmed by the next complete pull	High · 5–7 wk	M1, M2, M4	Shorter time from fail to pass, with a record linking every step (E5)
L4	As-of audit packages and a read-only auditor workspace, with evidence hashes and each result's method and rule version, sealed under a compliance-mode lock	High · 5–7 wk	M4, M5, M9; interviews with 2–3 audit firms first	Auditors can check the record without trusting CCP's database (C)
L5	Framework mapping proposals, as an offline batch first	Medium · 3–4 wk, plus reviewer hours	M9, a named compliance reviewer	New frameworks in days, with every mapping reviewed (E3)
L6	Published eval results: method, datasets, false-pass rate and pass^k (the chance that k repeated runs all succeed) for each AI workflow, refreshed each release	Medium · 2–3 wk, then each release	M8; L1 or L2 live	Buyers and auditors can check AI quality claims, which few vendors publish (D7)

#	Item	Complexity	Depends on	Customer value
L7	Connector and check drafting behind the conformance harness	Medium · 3–4 wk to set up, then per connector	M1, M9	Breadth grows without untested code (E2)

Release rule for anything that changes verdicts. Before the first customer, quick wins that change verdicts, such as Q5, Q8, Q9, Q10 and Q12, change them only on fixtures, so a before-and-after diff in CI is enough. From the first customer on, a change to a rule, the citation checker or a mapping ships as a new version. It comes with a preview of the verdicts it would change, and customers get notice before it takes effect (M9).

Not now

- A workflow server such as Temporal, a message broker such as Kafka, or a database per tenant. The hardened Postgres queue, or an engine that runs inside that database, comes first, and pooled tenancy under row-level security serves until a regulated customer needs its own deployment.
- A model deciding pass or fail, compliance status, connection health or identity matching.
- Agent or MCP write tools, or automatic approval of AI answers, before signed-in reviewers, the policy check and measured accuracy exist.
- A write role in customer clouds. Fixes go by pull request first; the companion document describes a narrow, opt-in role for customers who ask.
- New connectors in today's `pull()` shape, or a rule builder for customers, before the completeness record and the harness exist.
- Several agents that write, or one general agent with hundreds of tools.
- A compliance-mode lock on the raw zone: in that mode not even the root user can delete an object before its date (AWS). Standards exports such as FedRAMP 20x JSON or OSCAL wait for a regulated buyer.
- A rewrite. The raw zone, append-only observations, bitemporal state and extract-then-rule stay; the work rebuilds the edges around them.

G. Executive summary

CCP does not need more connectors or agents next quarter; it needs proof that the verdicts it already makes are right. The findings follow Section D's order, and the five investments below fix and measure first, then build what a first production customer needs.

Findings

1. **Live evidence cannot show a removal (D1).** The connector contract is one `pull()` method, with no connection lifecycle, cursor or record of whether a pull was complete (`ccp/connectors/base.py:49-54`). The live GitHub client emits snapshots only, and only removal events revoke a grant (`ccp/connectors/github/client.py:3-6`; `ccp/normalize/github_mapper.py:303`, `:321`). So a collaborator removed upstream stays `active`, and the remediation loop closes only in a test that injects the removal (`tests/e2e/test_remediation_loop.py:82-86`). Any error other than a 404 also aborts the pull mid-stream (`client.py:55-60`).
2. **The citation check can mark a false pass as verified (D2).** A quote passes if 85% of its words appear anywhere in the cited chunk, in any order (`ccp/evaluators/citations.py:36-47`), and the fields that decide pass or fail accept any quote that is present (`ccp/evaluators/extraction/__init__.py:30-40`). A probe on CCP's own fixture showed that a model answer of "completed", citing a training record that reads "In progress", verifies, and the HR-3 rule then passes (`fixtures/acme/hris/documents/training-E1044.md:12`; `ccp/evaluators/rules/hr3.py:51-69`). Tier 3 fixes retrieval quality at 1.0 (`ccp/agent/graphs/tier3.py:189`). No competitor documents citation checks in code, so this could be CCP's clearest lead; today it is worse than no check, because an auditor reads "verified" as proof.
3. **Approvals can fail open (D3).** `ccp approvals approve` accepts any `--decision` value, and the tier-3 gate treats anything but `override` or `regather` as approval, so `reject` or a typo approves the draft verdict (`ccp/approvals/cli.py:88-90`; `ccp/agent/graphs/tier3.py:231`, `:244-248`). Resume never compares the approved payload's hash with the checkpoint (`ccp/agent/tier3.py:234-240`), rollback re-grants access without the five gates (`ccp/remediation/runner.py:243-289`), and a reviewer is any typed name (`ccp/approvals/store.py:101-102`).
4. **Background work is safe only as one process (D4).** A verdict is committed, then announced in a second transaction, so a crash in between loses it (`ccp/evaluators/runner.py:93-106`). One bad record stalls a tenant's ordered topic (`ccp/ingest/consumers.py:61-72`). The Investigator reads such a topic, so a missing model key or a spent budget stops it the same way (`ccp/investigation/runner.py:82`, `:208-212`, `:237-238`). With several workers, rows can be skipped and a stale verdict can replace a fresh one (`ccp/ingest/event_log.py:58-66`; `ccp/compliance_state/writer.py:44-51`, `:67-78`). One $5 daily model budget per tenant is checked before the tier is known, so it also parks deterministic tests (`ccp/evaluators/runner.py:151-153`; `config/tenants.yaml:10`).

5. **Untrusted content and secrets are only partly contained (D5).** Injection handling looks for 15 fixed English phrases, and in tier 2 a hit only caps confidence (`ccp/security/prompt_guard.py:28-44`; `ccp/evaluators/confidence.py:76-78`). Tool results reach the model unfenced (`ccp/llm/client.py:126-140`). Chunk text goes unredacted to the embedding and rerank APIs (`ccp/retrieval/index.py:139`, `:151`; `ccp/retrieval/hybrid.py:231`). An upload manifest entry such as `../globex/report.pdf` is read from another tenant's folder and stamped with the wrong tenant (`ccp/connectors/manual/uploads.py:88`).
6. **Records cannot always be replayed or kept current (D6).** Replay looks chunks up in the live index, so it breaks once a cited document changes (`ccp/replay/rebuild.py:168-173`). Verdicts store no rule version (`ccp/evaluators/verdict.py:72-90`), and the impact table is code (`ccp/evaluators/impact.py:149-209`). Only full runs resolve identities and refresh the index (`ccp/compliance_state/run.py:65-67`), so on the change-driven path a GitHub account first seen after the last full run stays unlinked, and GH-1 and HR-1 can keep passing for a terminated employee until the next full run.
7. **AI quality is unmeasured (D7).** The repo has no git remote, so CI has never run, and all 27 recorded eval runs (`evals/runs/`, 12–15 Sep 2026) were keyless: no case used a token. The nightly gate blocks on any single pass-to-fail flip (`ccp/evals/regress.py:36-41`), which in our reading noise alone will trip on most nights. The judge runs on the tier-2 model, Haiku 4.5 (`ccp/config.py:73`, `:75`), whose earliest retirement date, 15 Oct 2026, has passed (Anthropic). Published eval numbers are rare in the category; Vanta and Sprinto publish some (Vanta, Jun 2026, Sprinto), so a measured false-pass rate would set CCP apart.
8. **There is no product surface or operating model (D8).** CCP is teaching-scale by its own spec (`spec.md:1`). It is a command line with two fixture tenants in YAML, worker processes that each poll one consumer, and a raw zone on local disk (`pyproject.toml:12`; `config/tenants.yaml:1-3`; `ccp/ingest/worker.py:67-103`; `ccp/config.py:49`, `:133-135`). It has no API, UI, sign-in, roles, onboarding, deployment or purge path, and `spec.md` line 3 still says no code exists.
9. **Competitors lead on breadth and surface; CCP can lead on proof (B, C).** Vanta claims 400+ integrations, against CCP's three connectors, one of them live (B.1, A). Yet none of the six incumbents documents an immutable event ledger, bitemporal state or citation checks in code, no AI-native entrant publishes a citation-verification step, and no incumbent documents what a partial sync from a native connector does. Most incumbents let agents act with the user's permissions over MCP, while CCP's agents only read and propose (A.4). After the 2026 allegations against Delve, which Delve denies, results a buyer can check are an open position, in our reading.

Invest in these first

#	Investment	Effort	Why first
1	Fail closed: a private remote with CI, approvals, upload paths, verdict writes, dead letters for ordered topics, and a model budget that never parks tier 1 (Q1–Q4, Q6, Q7)	4.5–6 engineer-weeks	Small fixes; today a typo approves, a crash can lose a verdict, and one bad record stops a tenant's topic
2	Citations that test support, measured on real models (Q5, Q13)	5–7 engineer-weeks, plus a reviewer's labelling time	No AI-tier verdict should count until "verified" means supported and the false-pass rate is known
3	Contain untrusted content and secrets, and keep records current and replayable (Q8–Q11)	4.5–6.5 engineer-weeks	Needed before real customer files arrive; every verdict stays explainable after a document or rule changes
4	Live evidence a customer can trust: hardened GitHub pulls, then the connection lifecycle, completeness record and conformance harness (Q12, M1)	9–12 engineer-weeks	Live mode cannot see a removal today, and more connectors in today's shape would multiply that failure
5	Durable execution, a typed tool registry, and an API with signed-in users and roles (M2, M3, M4)	14–20 engineer-weeks	Work survives crashes and deploys, and every approval has an accountable reviewer

Together that is about 37–52 engineer-weeks, all estimates: roughly 19 to 26 weeks, or four to six months, for three engineers who spend about two-thirds of their time on this plan (about 2 engineer-weeks per calendar week). It includes every quick win, which together address each P0 gap in Section D, the work Gate 1 checks. The rest of the medium term (M5–M9, about 18–24 engineer-weeks) covers the remaining Gate 2 items, so a first production customer is about six to nine months away at this capacity. The differentiators follow: questionnaire answers with verified citations (E1, L1), evidence review of more upload types (E4, L2) and remediation by pull request (E5, L3).

Decisions needed

- Leadership: confirm the capacity the plan assumes, three engineers at about two-thirds time, and name an owner for each workstream. Hold new connectors until the completeness record and the conformance harness exist (D1, F).
- Compliance: name a domain expert to label evidence for GH-4, HR-3 and HR-4, starting with about 1–2 weeks of labelling, and agree the false-pass limit that Gate 1 requires (D2, D7, Q13).
- Security and legal: before real customer data arrives, settle retention terms with each model provider and opt out of Voyage model training (Q9). Agree retention and erasure terms for customer data before any retention lock is set (M5, D8).
- AI: pin dated model ids now (Q13). Choose the judge's model family, which adds a vendor and its data terms, and choose tier 2's next model by a paired comparison before a retirement notice for Haiku 4.5 forces the move (D7, M8).
- Engineering: choose where the private remote lives and turn on CI this week (Q1). Agree the crash and deploy tests that decide whether the hardened Postgres queue is enough or CCP moves to DBOS Transact (M2).
- Product: choose the first customer's systems for M7: AWS through a read role, an identity provider, and HRIS data through an aggregator, which adds a vendor that handles customer HR data. Confirm that fixes go by pull request first, with no write role in customer clouds for now (E5, F).

Sources

The `agentic` repository was read at `3386899` on 9 Nov 2026 without changes, fetches or branch switches. Every external page below was opened by the research behind this report on 9 Nov 2026, not taken from search snippets, and links to lines of open-source code are pinned to the commit that was read. Load-bearing claims went through separate read-only passes that tried to refute them; the text reflects their corrections.

Internal evidence

Source	Ref read	Used for
`agentic` repository (Agentic CCP): `ccp/` with its 15 SQL migrations, `tests/`, `fixtures/`, `config/`, `evals/` and `.github/workflows/`	Commit `3386899` (12 Sep 2026); the repo has no git remote	A, C–G
`spec.md` v1.3, in the same commit	`3386899`	A, D, F, G

Source	Ref read	Used for
Local eval runs in `evals/runs/`	27 run files from 12–15 Sep 2026; git-ignored (`.gitignore:4`), so they are in no commit	A.6, C, D7, G
Companion document, Agentic CCP: Target Architecture for AI-Native Compliance	9 Nov 2026 version, written from the same evidence	Scope; E and F cite its Sections 3.5, 4.3, 4.4 and 6.2

Competitor, standards and provider pages

Source	Pages
Vanta	Tests, Resources, Build Integrations overview, Test snapshot API, Hosted MCP, MCP FAQ, Remediate with MCP, Automated test history, Custom tests, Custom agents, Product updates, Access reviews, Common controls, Merge, What's new, Verifiers, AI quality maturity model, Agent capabilities, TPRM Agent, AARM donation, Database limits, Inc-868 RCA, Trust updates, Subprocessors, Compare page, Integrations job, AI platform job
Drata	Monitoring, Test details, Advanced editor, AI summaries, Framework Mapping Agent, TPRM Agent, Access reviews, Audit sampling, Auditor access, Workday matching, Compliance as Code, Custom connections, MCP server, MCP OAuth metadata, AI-generated tests, Harness engineering, Securing data at scale, Subprocessors, Interoperability job, SafeBase AIQA guide
Secureframe	Managing connections, Custom automated tests, Test activity, Comply AI remediation, AI evidence validation, MCP server, Custom integrations, FedRAMP 20x Moderate, Subprocessors
Thoropass	Evidence reuse, Connected documents, Monitors, Smart Sort, MCP server, MCP OAuth metadata, MCP post, Platform, Newsroom, PM job

Source	Pages
Sprinto	Get started with ingestion, Ingestion guide, Entity types, Custom integrations, Aggregators, Wiz ingestion, Monitor FAQ, Automated checks, Fix-it extension, MCP, Aug 2026 updates, Subprocessors
Scrut	Tests, Scan status, 15Five connector, Access reviews, Teammates, MCP tools, Sep 2026 updates, Open API, Subprocessors
AI-native entrants	Delve (allegations, YC split, site), Comp AI (repo, commits, funding), Probo, Anecdotes, Zania (site, funding), Complyance, Fieldguide
Open-source code, pinned to the commit read	Comp AI at `150fed5` (reconciliation, remediation, AWS command executor, check storage schema); Probo at `7bbe06a` (connector capabilities, cassette safety test, approvals, agent setup, checkpoints, execution service, execution handler, approval policies, MCP resolver); CISO Assistant at `a867086` (tool-selection harness)
Standards	FedRAMP 20x, FedRAMP 2026 certification rules, FedRAMP RFC-0024, Lula 2, OSCAL releases
Model and cloud providers	Anthropic (Model IDs and versioning, Model deprecations, which F links as its Markdown page), Voyage AI (model-training opt-out, in MongoDB's docs), AWS (S3 Object Lock)